

计算机组成原理

讲师：老汤

加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】

第一章：计算机系统概述

第二章：总线系统

第三章：主存储器

第四章：数据的表示与运算

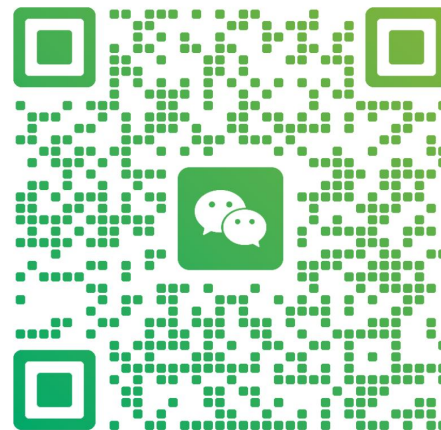
✓ 第五章：指令系统

第六章：中央处理器（CPU）

第七章：输入输出（I/O）系统



老汤
浙江 杭州



扫一扫上面的二维码图案，加我为朋友。

X86-64 指令系统 (复杂指令集, CISC)

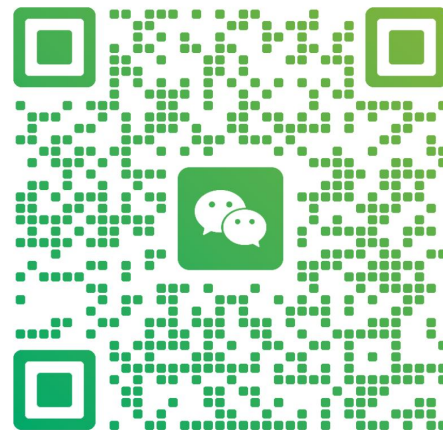
加老汤微信, 帮你 408 冲 120+
咨询【老汤 408 高分特训营】

两种学习方式 {
godbolt 网站
objdump 命令

X86-64 两种汇编代码格式 {
intel 格式
ATT 格式



老汤
浙江 杭州



扫一扫上面的二维码图案, 加我为朋友。

X86-64 指令系统

gcc 命令

objdump 命令



X86-64 指令系统

objdump -d -M intel test.exe

```
000000000040052d <main>:
40052d: 55                push    rbp
40052e: 48 89 e5          mov     rbp, rsp
400531: 48 83 ec 10       sub     rsp, 0x10
400535: c7 45 fc 03 00 00 00 mov     DWORD PTR [rbp-0x4], 0x3
40053c: c7 45 f8 02 00 00 00 mov     DWORD PTR [rbp-0x8], 0x2
400543: 8b 55 f8          mov     edx, DWORD PTR [rbp-0x8]
400546: 8b 45 fc          mov     eax, DWORD PTR [rbp-0x4]
400549: 89 d6            mov     esi, edx
40054b: 89 c7            mov     edi, eax
40054d: e8 b1 ff ff ff   call    400503 <_add>
400552: 89 45 f4          mov     DWORD PTR [rbp-0xc], eax
400555: b8 00 00 00 00   mov     eax, 0x0
40055a: c9              leave
40055b: c3              ret
```

X86-64 指令系统（复杂指令集，CISC）

① 指令系统包含哪些内容

② 指令系统的设计原则

③ RISC 和 CISC 的区别

抽象：如何设计一个指令系统？

具体的指令系统实例：

MIPS

X86-64

三部分内容

寄存器组

指令格式

寻址方式

程序的机器级表示

选择结构

循环结构

函数调用

X86 指令系统：寄存器组

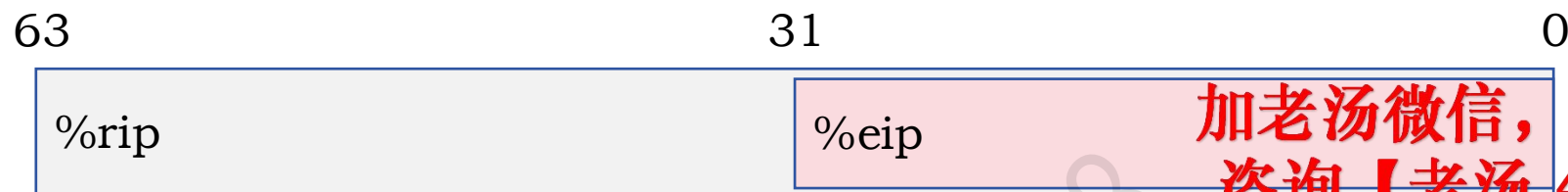
最初的 8086 → IA32 → x86-64

63	31	15	7	0	
%rax	%eax	%ax	%al		返回值
%rbx	%ebx	%bx	%bl		
%rcx	%ecx	%cx	%cl		第 4 个参数
%rdx	%edx	%dx	%dl		第 3 个参数
%rsi	%esi	%si	%sil		第 2 个参数
%rdi	%edi	%di	%dil		第 1 个参数
%rbp	%ebp	%bp	%bpl		
%rsp	%esp	%sp	%spl		栈指针 (stack pointer)

X86-64 指令系统：寄存器组

63	31	15	7	0	
%r8	%r8d	%r8w	%r8b		第 5 个参数
%r9	%r9d	%r9w	%r9b		第 6 个参数
%r10	%r10d	%r10w	%r10b		
%r11	%r11d	%r11w	%r11b		
%r12	%r12d	%r12w	%r12b		
%r13	%r13d	%r13w	%r13b		
%r14	%r14d	%r14w	%r14b		
%r15	%r15d	%r15w	%r15b		

X86-64 指令系统：寄存器组



加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】

RIP 是指令指针寄存器 (Instruction Pointer Register)

其实就是程序计数器 PC 的实现



老汤
浙江 杭州



扫一扫上面的二维码图案，加我为朋友。

X86-64 指令系统：数据大小（字）

C 声明数据类型	大小	Intel x86-64 数据类型
char	1 字节	字节
short	2 字节	字 (word)
int	4 字节	双字 (double words)
long	8 字节	四字 (quad words)
char*	8 字节	四字 (quad words)
float	4 字节	单精度
double	8 字节	双精度

X86 指令系统：寄存器组

最初的 8086 → IA32 → x86-64

63	31	15	7	0	
%rax	%eax	%ax	%al		返回值
%rbx	%ebx	%bx	%bl		
%rcx	%ecx	%cx	%cl		第 4 个参数
%rdx	%edx	%dx	%dl		第 3 个参数
%rsi	%esi	%si	%sil		第 2 个参数
%rdi	%edi	%di	%dil		第 1 个参数
%rbp	%ebp	%bp	%bpl		
%rsp	%esp	%sp	%spl		栈指针 (stack pointer)

X86-64 指令系统：数据传送指令

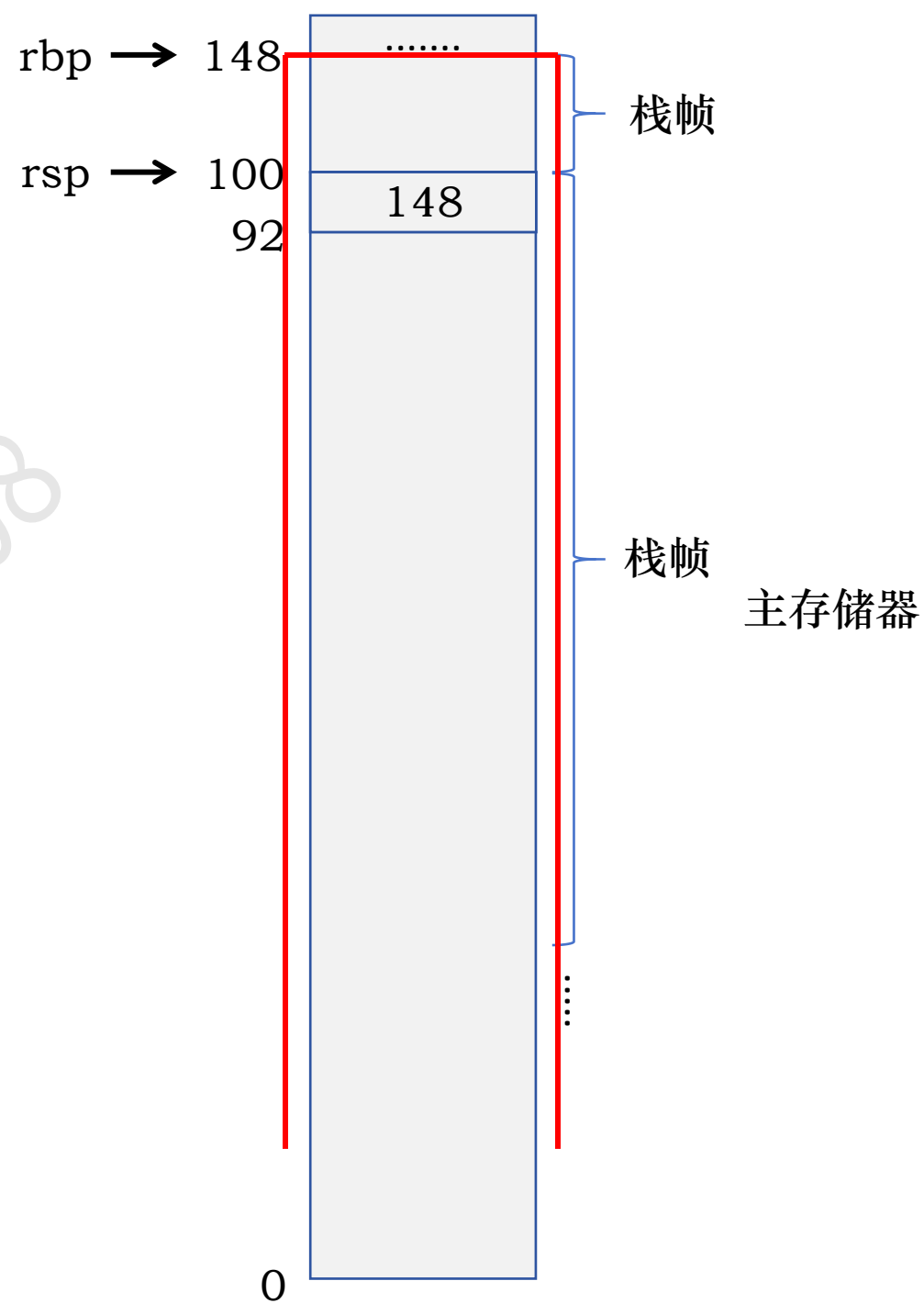
```
int main() {  
    char d = 'f';  
    short b = 10;  
    int a = 1000;  
    long c = 3000000;  
    char* p = &d;  
    return 0;  
}
```

main:

→ push rbp
mov rbp, rsp
mov BYTE PTR [rbp-25], 102
mov WORD PTR [rbp-2], 10
mov DWORD PTR [rbp-8], 1000
mov QWORD PTR [rbp-16], 3000000
lea rax, [rbp-25]
mov QWORD PTR [rbp-24], rax
mov eax, 0
pop rbp
ret

① $R[rs] \leftarrow R[rs] - 8$

② $M[R[rs]] \leftarrow R[rbp]$



X86-64 指令系统：数据传送指令

```
int main() {  
    char d = 'f';  
    short b = 10;  
    int a = 1000;  
    long c = 3000000;  
    char* p = &d;  
    return 0;  
}
```

lea: load effective address 的缩写

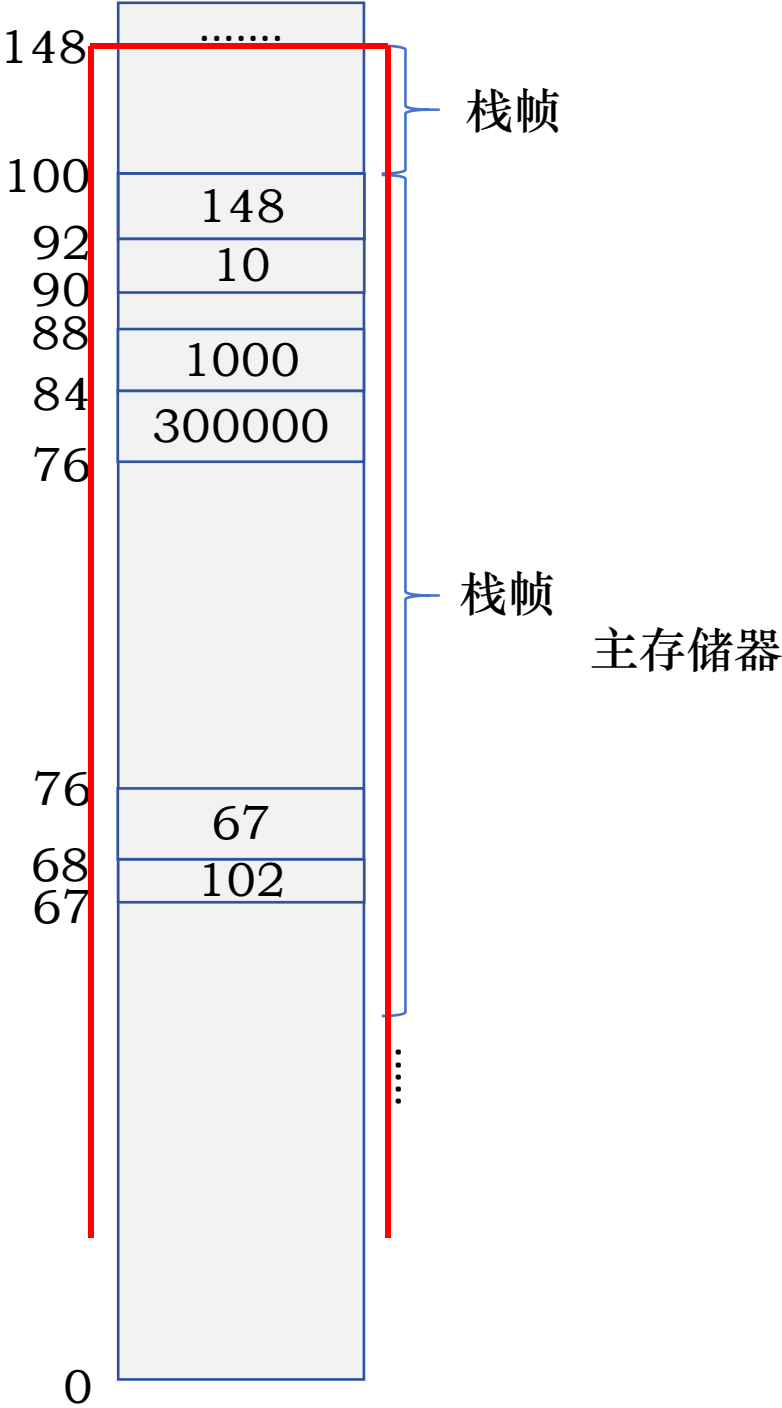
加载有效内存地址指令

main:

```
push    rbp  
mov     rbp, rsp  
→ mov   BYTE PTR [rbp-25], 102  
mov     WORD PTR [rbp-2], 10  
mov     DWORD PTR [rbp-8], 1000  
mov     QWORD PTR [rbp-16], 3000000  
lea     rax, [rbp-25]  
mov     QWORD PTR [rbp-24], rax  
mov     eax, 0  
pop     rbp  
ret
```

rbp → rsp →

%rax: 67



X86-64 指令系统：数据传送指令

```
int main() {  
    char d = 'f';  
    short b = 10;  
    int a = 1000;  
    long c = 3000000;  
    char* p = &d;  
    return 0;  
}
```

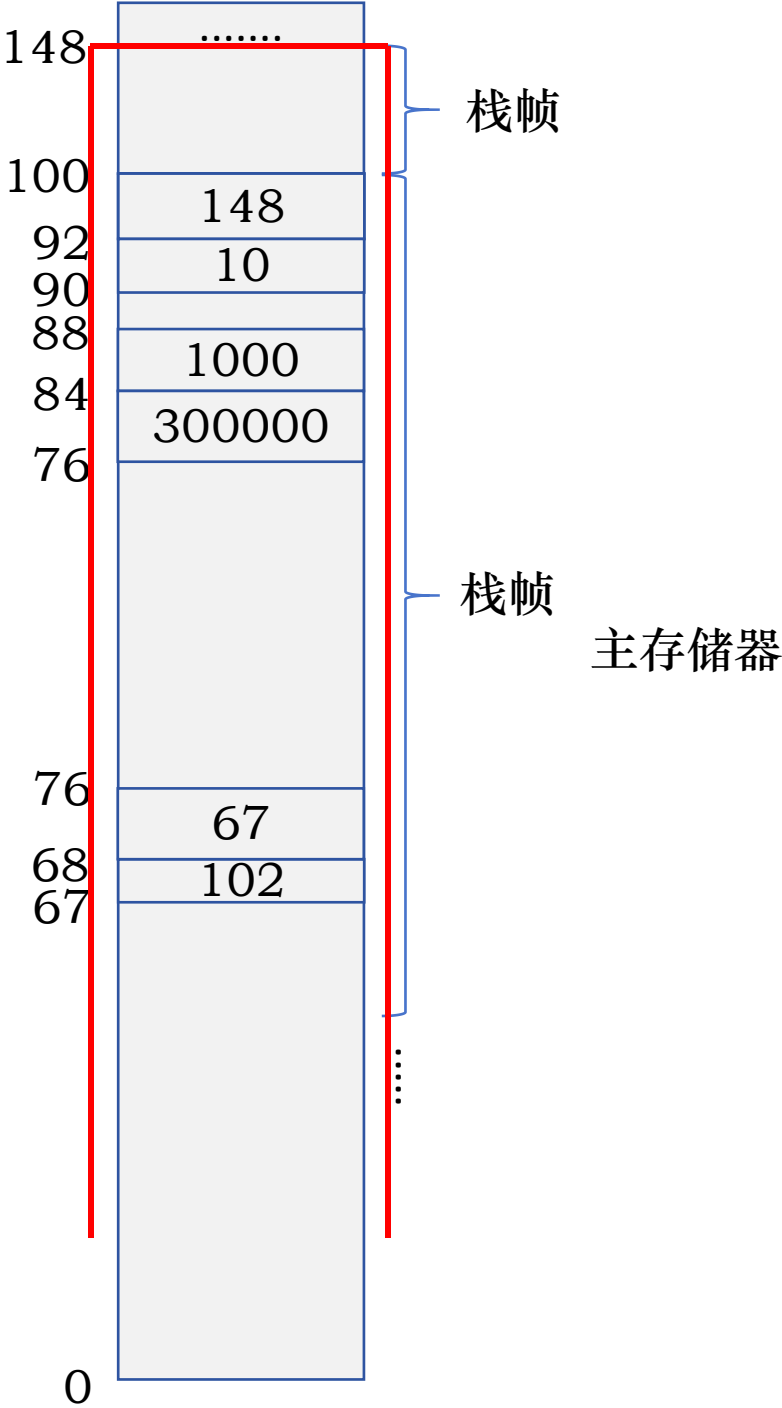
- ① $R[rbp] \leftarrow M[R[rsi]]$
- ② $R[rsi] \leftarrow R[rsi] + 8$

main:

```
push    rbp  
mov     rbp, rsi  
mov     BYTE PTR [rbp-25], 102  
mov     WORD PTR [rbp-2], 10  
mov     DWORD PTR [rbp-8], 1000  
mov     QWORD PTR [rbp-16], 3000000  
lea     rax, [rbp-25]  
mov     QWORD PTR [rbp-24], rax  
mov     eax, 0  
→ pop   rbp  
ret
```

rbp → rsi →

%rax: 67



X86-64 指令系统：数据传送指令

① push 和 pop

② mov

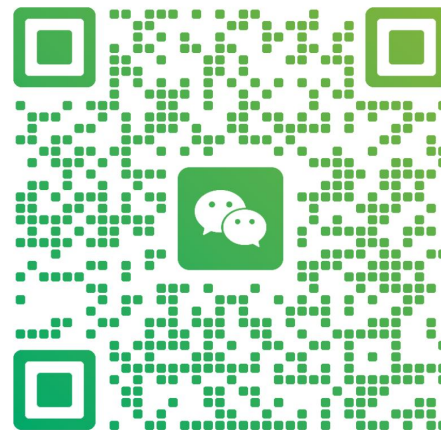
③ lea

加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】



老汤

浙江 杭州



扫一扫上面的二维码图案，加我为朋友。

X86-64 指令系统：数据传送指令 (push 和 pop)

push S 将 S 中的四字数据压入栈内存

push rax

① $R[rsi] \leftarrow R[rsi] - 8$

② $M[R[rsi]] \leftarrow R[rax]$

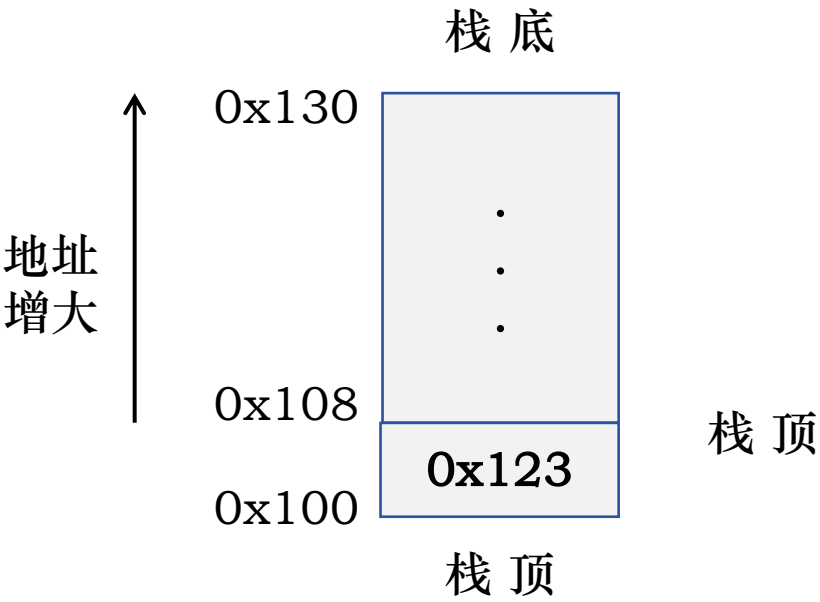
pop D 弹出栈顶四字数据，传送到 D 的位置

pop rdx

① $R[rdi] \leftarrow M[R[rsi]]$

② $R[rsi] \leftarrow R[rsi] + 8$

%rax	0x123
%rdi	0x123
%rsi	0x100



堆栈操作指令： 寄存器寻址 隐含寻址 另一个操作数隐含为栈顶元素，指令中无须明显指出操作数地址

X86-64 指令系统：数据传送指令（mov 指令）

mov D, S D←S

main:

```
push    rbp
mov     rbp, rsp
mov     BYTE PTR [rbp-25], 102
mov     WORD PTR [rbp-2], 10
mov     DWORD PTR [rbp-8], 1000
mov     QWORD PTR [rbp-16], 300000
lea     rax, [rbp-25]
mov     QWORD PTR [rbp-24], rax
mov     eax, 0
pop     rbp
ret
```

寄存器 → 寄存器

立即数 → 主存储器

寄存器 → 主存储器

立即数 → 寄存器

主存储器 → 寄存器

mov rax, WORD PTR [rbp - 12]



主存储器 → 主存储器

立即数寻址 寄存器寻址 基址寻址

X86-64 指令系统：数据传送指令 (lea 指令)

lea: load effective address 的缩写

```
void swap(int v[], int k) {
    int temp;
    temp = v[k];
    v[k] = v[k + 1];
    v[k + 1] = temp;
}
```

加载有效内存地址指令

lea D, S $D \leftarrow \&S$

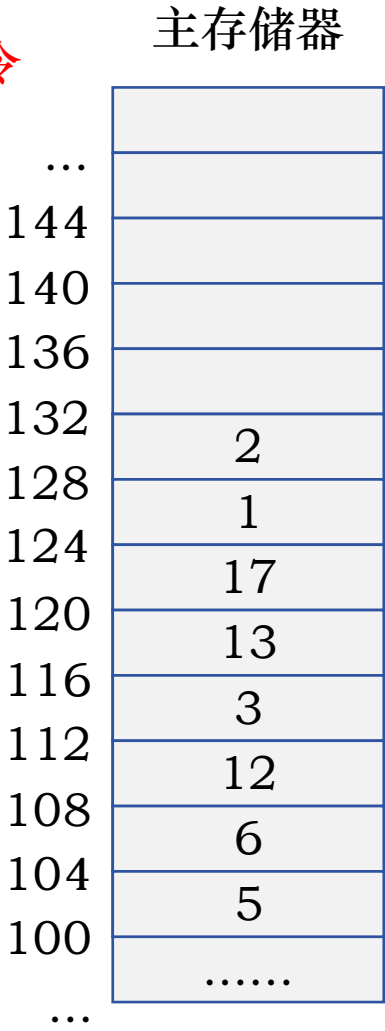
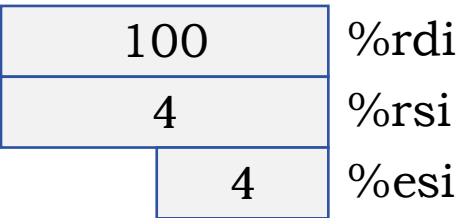
movsx: move with sign - extend 的缩写，符号扩展指令

swap:

```
→ movsx    rsi, esi
   lea      rdx, [rdi+rsi*4]
   mov      ecx, DWORD PTR [rdx]
   lea      rax, [rdi+4+rsi*4]
   mov      esi, DWORD PTR [rax]
   mov      DWORD PTR [rdx], esi
   mov      DWORD PTR [rax], ecx
   ret
```

对 k 进行符号扩展到 64 位

数组 v 基地址
符号扩展的变量 k
变量 k



X86-64 指令系统：数据传送指令 (lea 指令)

lea: load effective address 的缩写

```
void swap(int v[], int k) {  
    int temp;  
    temp = v[k];  
    v[k] = v[k + 1];  
    v[k + 1] = temp;  
}
```

lea D, S D ← &S

加载有效内存地址指令

100 + 4 * 4 = 116

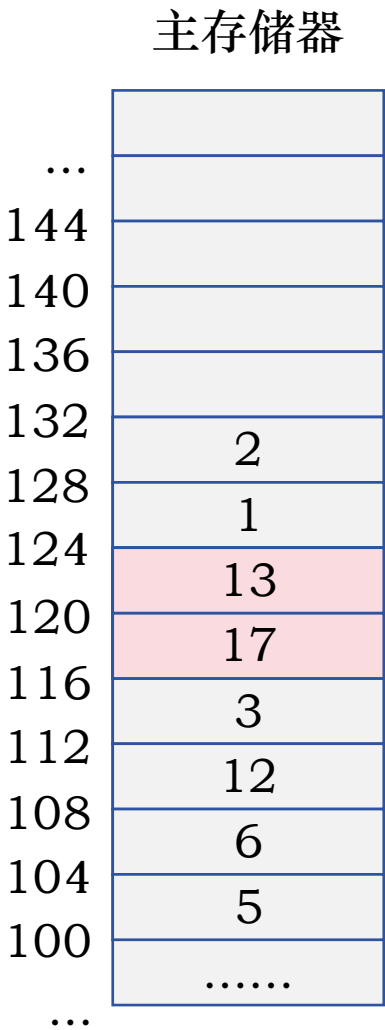
swap:

```
→ movsx    rsi, esi  
   lea      rdx, [rdi+rsi*4]  
   mov      ecx, DWORD PTR [rdx]  
   lea      rax, [rdi+4+rsi*4]  
   mov      esi, DWORD PTR [rax]  
   mov      DWORD PTR [rdx], esi  
   mov      DWORD PTR [rax], ecx  
   ret
```

对 k 进行符号扩展到 64 位
将 v[k] 的内存地址放到 rdx
将 v[k] 赋值给 temp
将 v[k+1] 的内存地址放到 rax
将 v[k+1] 复制到 esi
将 v[k+1] 复制给 v[k]
将 temp 复制给 v[k+1]

数组 v 基地址
符号扩展的变量 k
下标 k + 1 对应元素
下标 k 对应元素 (temp 变量)

120	%rax
116	%rdx
100	%rdi
4	%rsi
17	%esi
13	%ecx



X86-64 指令系统：数据传送指令 (lea 指令)

lea: load effective address 的缩写

```
void swap(int v[], int k) {
    int temp;
    temp = v[k];
    v[k] = v[k + 1];
    v[k + 1] = temp;
}
```

```
swap:
    movsx    rsi, esi
    lea      rdx, [rdi+rsi*4]
    mov      ecx, DWORD PTR [rdx]
    lea      rax, [rdi+4+rsi*4]
    mov      esi, DWORD PTR [rax]
    mov      DWORD PTR [rdx], esi
    mov      DWORD PTR [rax], ecx
    ret
```

数组基地址 (基址寄存器)

数组下标 (变址寄存器)

1、2、4 或者 8

$M[R[rdi] + R[rsi] * s]$

比例变址寻址方式

数组 v 基地址
符号扩展的变量 k
下标 k + 1 对应元素
下标 k 对应元素 (temp 变量)

120	%rax
116	%rdx
100	%rdi
4	%rsi
17	%esi
13	%ecx

主存储器	
...	
144	
140	
136	
132	2
128	1
124	13
120	17
116	3
112	12
108	6
104	5
100	
...	

X86-64 指令系统：数据传送指令

① push 和 pop

② mov

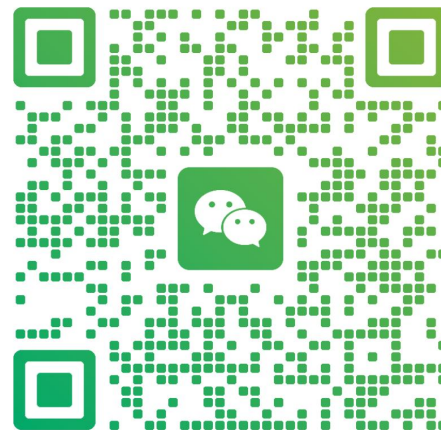
③ lea

加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】



老汤

浙江 杭州



扫一扫上面的二维码图案，加我为朋友。

X86-64 指令系统：算术和逻辑运算指令

指令	效果	描述
lea D, S	$\&S \rightarrow D$	加载有效地址
inc D dec D neg D not D	$D + 1 \rightarrow D$ $D - 1 \rightarrow D$ $-D \rightarrow D$ $\sim D \rightarrow D$	加 1 减 1 取负 取补
add D, S sub D, S imul D, S xor D, S or D, S and D, S	$D + S \rightarrow D$ $D - S \rightarrow D$ $D * S \rightarrow D$ $D \wedge S \rightarrow D$ $D \mid S \rightarrow D$ $D \& S \rightarrow D$	加 减 乘 异或 或 与
sal D, k shl D, k sar D, k shr D, k	$D \ll k \rightarrow D$ $D \ll k \rightarrow D$ $D \gg k \rightarrow D$ $D \gg k \rightarrow D$	左移 左移 (等同于 sal) 算术右移 逻辑右移

X86-64 指令系统：算术和逻辑运算指令 (lea 指令)

lea D, S $D \leftarrow \&S$

```

                rdi      rsi      rdx
                ↑        ↑        ↑
long scale(long x, long y, long z) {
    long t = x + 4 * y + 12 * z;  → (x + y * 4) + (z + z * 2) * 4
    return t;
}
```

scale:

```
→ lea    rax, [rdi+rsi*4]
   lea    rdx, [rdx+rdx*2]
   lea    rax, [rax+rdx*4]
   ret
```

$rax \leftarrow x + y * 4$

$rdx \leftarrow z + z * 2$

$rax \leftarrow (x + y * 4) + (z + z * 2) * 4$

$x + 4 * y + 12 * z$

X86-64 指令系统：算术和逻辑运算指令 - add 指令

```
int add(int a, int b) {  
    int c = a + b * 3;  
    return c;  
}
```

Diagram showing the mapping of C parameters to registers:
- `a` is mapped to `rdi`
- `b` is mapped to `rsi`

↓

```
add:  
    lea    eax, [rsi+rsi*2]  
    add    eax, edi  
    ret
```

加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】



老汤
浙江 杭州



扫一扫上面的二维码图案，加我为朋友。

X86 指令系统：寄存器组

最初的 8086 → IA32 → x86-64

63	31	15	7	0	
%rax	%eax	%ax	%al		返回值
%rbx	%ebx	%bx	%bl		
%rcx	%ecx	%cx	%cl		第 4 个参数
%rdx	%edx	%dx	%dl		第 3 个参数
%rsi	%esi	%si	%sil		第 2 个参数
%rdi	%edi	%di	%dil		第 1 个参数
%rbp	%ebp	%bp	%bpl		
%rsp	%esp	%sp	%spl		栈指针 (stack pointer)

X86-64 指令系统：算术和逻辑运算指令 - add 指令

加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】

```
int add(int a, int b) {  
    int c = a + b * 3;  
    return c;  
}
```

↑ rdi ↑ rsi

↓

```
add:  
    lea    eax, [rsi+rsi*2]  
    add    eax, edi  
    ret
```

寄存器寻址



扫一扫上面的二维码图案，加我为朋友。

X86-64 指令系统：算术和逻辑运算指令 - add 指令

```
void add_ele(int A[], int h) {  
    A[4] = h + A[3];  
}
```

Diagram showing the mapping of C code to assembly:
- `A[]` is mapped to `rdi` (indicated by an arrow).
- `h` is mapped to `esi` (indicated by an arrow).

`add rsp, 8`

`add_ele:`

```
add    esi, DWORD PTR [rdi+12]  
mov    DWORD PTR [rdi+16], esi  
ret
```

`add <reg>, <reg>`

`add <reg>, <mem>`

`add <mem>, <reg>`

`add <reg>, <imme>`

`add <mem>, <imme>`

add 指令操作数可以是立即数、也可以是寄存器中的数据、也可以是主存中的数据

X86-64 指令系统：算术和逻辑运算指令 - add 指令

只能对寄存器中的操作数进行算术和逻辑运算

通过 load 和 store 指令

```
void add_ele(int A[], int h) {  
    A[4] = h + A[3];  
}
```

rdi esi

```
void add_ele(int A[], int h) {  
    A[4] = h + A[3];  
}
```

\$4 \$5

add_ele:

```
    add    esi, DWORD PTR [rdi+12]  
    mov    DWORD PTR [rdi+16], esi  
    ret
```

add_ele:

```
    lw     $2, 12($4)  
    nop  
    addu   $2, $2, $5  
    sw     $2, 16($4)  
    jr     $31  
    nop
```

x86-64 复杂指令集 (CISC)

load/store 体系结构 MIPS 精简指令集 (RISC)

可以对内存和寄存器中的操作数进行算术逻辑运算

X86-64 指令系统：算术和逻辑运算指令

指令	效果	描述
lea D, S	$\&S \rightarrow D$	加载有效地址
inc D dec D neg D not D	$D + 1 \rightarrow D$ $D - 1 \rightarrow D$ $-D \rightarrow D$ $\sim D \rightarrow D$	加 1 减 1 取负 取补
add D, S sub D, S imul D, S xor D, S or D, S and D, S	$D + S \rightarrow D$ $D - S \rightarrow D$ $D * S \rightarrow D$ $D \wedge S \rightarrow D$ $D \mid S \rightarrow D$ $D \& S \rightarrow D$	加 减 乘 异或 或 与
sal D, k shl D, k sar D, k shr D, k	$D \ll k \rightarrow D$ $D \ll k \rightarrow D$ $D \gg k \rightarrow D$ $D \gg k \rightarrow D$	左移 左移 (等同于 sal) 算术右移 逻辑右移

lea 指令除外，其他所有的 D 和 S 既可以是寄存器名称，也可以是主存地址

X86-64 指令系统：算术和逻辑运算指令

指令	效果	描述
lea D, S	$\&S \rightarrow D$	加载有效地址
inc D dec D neg D not D	$D + 1 \rightarrow D$ $D - 1 \rightarrow D$ $-D \rightarrow D$ $\sim D \rightarrow D$	加 1 减 1 取负 取补
add D, S sub D, S imul D, S xor D, S or D, S and D, S	$D + S \rightarrow D$ $D - S \rightarrow D$ $D * S \rightarrow D$ $D \wedge S \rightarrow D$ $D \mid S \rightarrow D$ $D \& S \rightarrow D$	加 减 乘 异或 或 与
sal D, k shl D, k sar D, k shr D, k	$D \ll k \rightarrow D$ $D \ll k \rightarrow D$ $D \gg k \rightarrow D$ $D \gg k \rightarrow D$	左移 左移 (等同于 sal) 算术右移 逻辑右移

编译器对指令优化的重要性

X86-64 指令系统：算术和逻辑运算指令

sal: shift arithmetic left 的缩写, 算术左移指令

sar: shift arithmetic right 的缩写, 算术右移指令

```
long shift_left2_rightn(long x, long n) {
    → x <<= 4;
    x >>= n;
    return x;
}
```

```
shift_left2_rightn:
```

```
→ mov     rax, rdi
   sal     rax, 4
   mov     ecx, esi
   sar     rax, cl
   ret
```



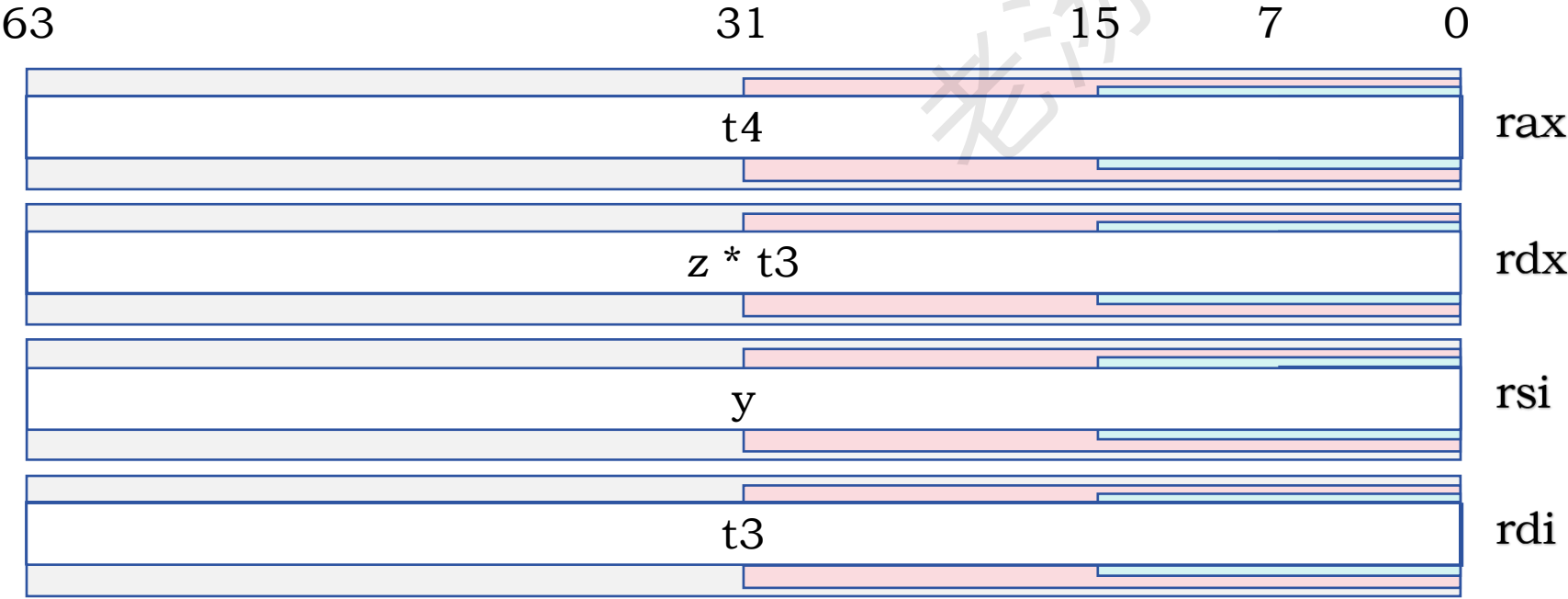
X86-64 指令系统：算术和逻辑运算指令

arith:

```
→ xor    rdi, rsi
  and     edi, 252645135
  imul    rdx, rdi
  lea     rax, [rdx+rdx*2]
  sal     rax, 4
  ret
```

```
rdi      rsi      rdx
  ↑       ↑       ↑
long arith(long x, long y, long z) {
→ long t1 = x ^ y;
  long t2 = z * 48;
  long t3 = t1 & 0xF0F0F0F;
  long t4 = t2 * t3;
  return t4;
}
```

$$z * 48 * t3 = (z * t3 + 2 * z * t3) << 4$$
$$= (rdx + 2 * rdx) << 4$$



X86-64 指令系统：算术和逻辑运算指令

edi

esi

rdx

rcx

↑

↑

↑

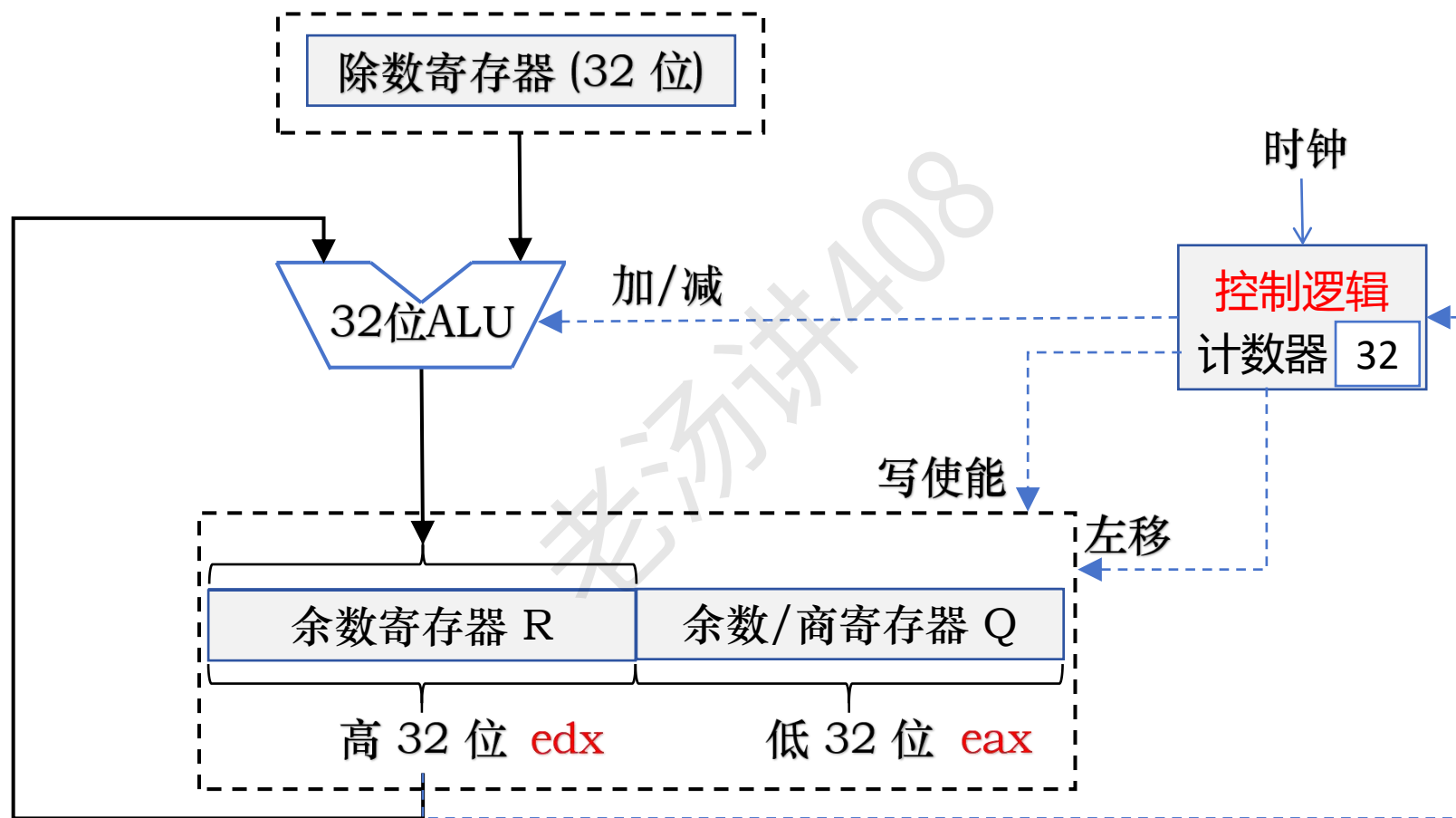
↑

```
void remdiv(int x, int y, int *qp, int *rp) {  
    → int q = x / y;  
    int r = x % y;  
    *qp = q;  
    *rp = r;  
}
```

```
remdiv:  
    → mov    eax, edi  
      mov    r8, rdx  
      cdq  
      idiv   esi  
      mov    DWORD PTR [r8], eax  
      mov    DWORD PTR [rcx], edx  
      ret
```

r8	*qp
rcx	*rp
rdx	rp
eax	x
esi	y
edi	x

定点数除法运算：原码除法运算（电路优化）



X86-64 指令系统：算术和逻辑运算指令

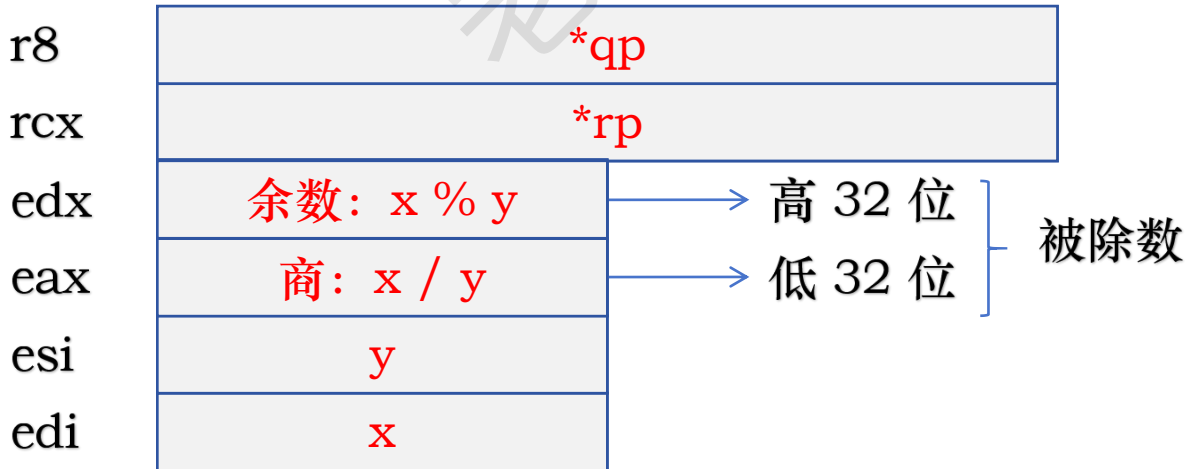
cdq: Convert Doubleword to Quadword
的缩写，将双字扩展成四字指令

edi esi rdx rcx

↑ ↑ ↑ ↑

```
void remdiv(int x, int y, int *qp, int *rp) {  
→ int q = x / y;  
  int r = x % y;  
  *qp = q;  
  *rp = r;  
}
```

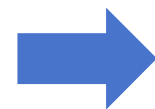
```
remdiv:  
→ mov     eax, edi  
  mov     r8, rdx  
  cdq  
  idiv    esi  
  mov     DWORD PTR [r8], eax  
  mov     DWORD PTR [rcx], edx  
  ret
```



X86-64 指令系统：选择语句 (if...else...)

cmp: compare 的缩写，比较指令 rdi - rsi

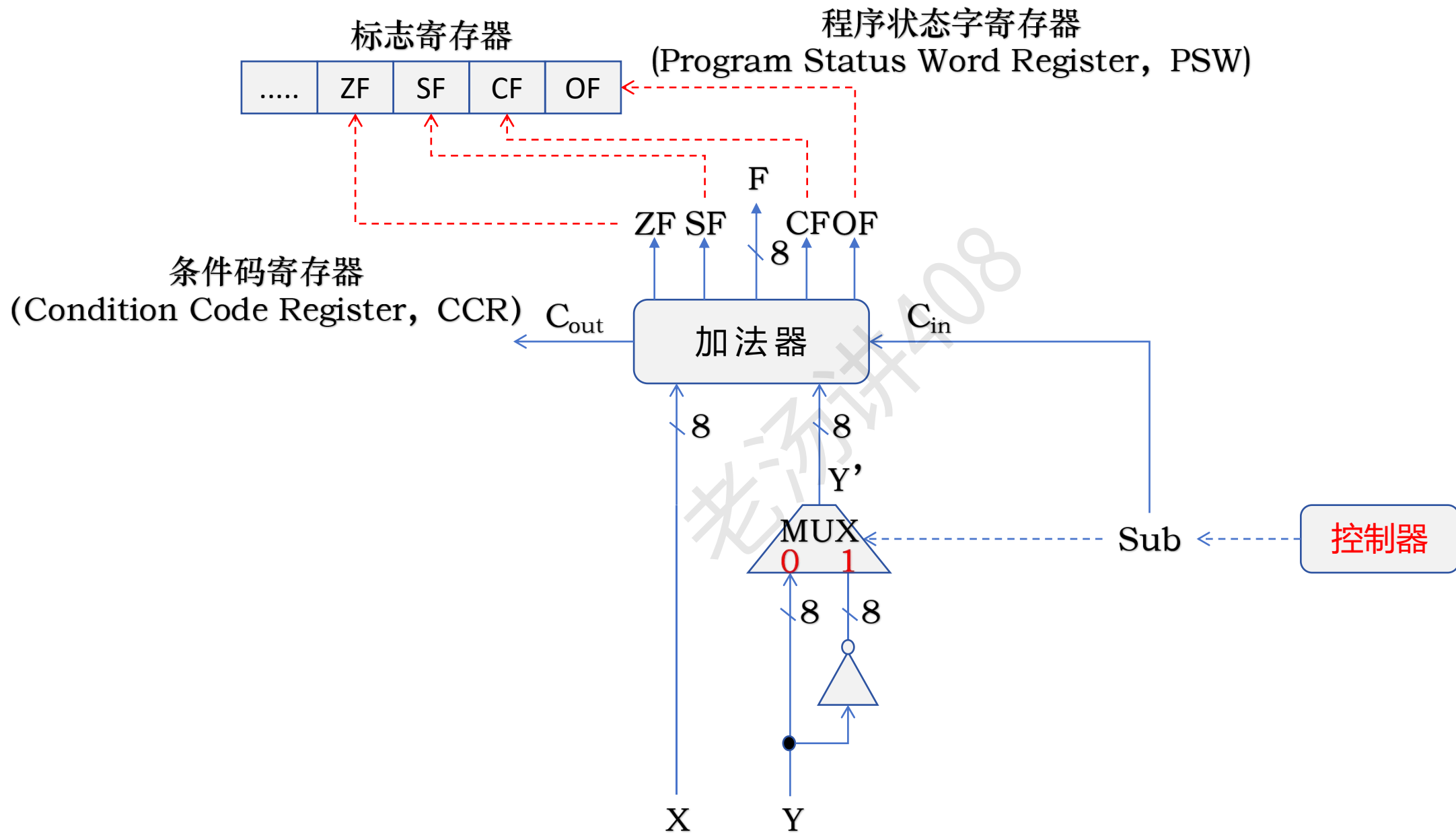
```
long lt_cnt = 0;
long ge_cnt = 0;
long absdiff_se(long x, long y) {
    long result;
    if (x < y) {
        lt_cnt++;
        result = y - x;
    } else {
        ge_cnt++;
        result = x - y;
    }
    return result;
}
```



```
absdiff_se:
    → cmp     rdi, rsi
    jge      .L2
    add      QWORD PTR lt_cnt[rip], 1
    mov      rax, rsi
    sub      rax, rdi
    ret

.L2:
    add      QWORD PTR ge_cnt[rip], 1
    mov      rax, rdi
    sub      rax, rsi
    ret
```

带标志的加法器



X86-64 指令系统：选择语句 (if...else...)

```
long lt_cnt = 0;
long ge_cnt = 0;
long absdiff_se(long x, long v) {
```



指令	同义名	跳转条件	描述
jmp Label		1	直接跳转
jmp *Operand		1	间接跳转
je Label	jz	ZF	相等/零
jne Label	jnz	~ZF	不相等/非零
js Label		SF	负数
jns Label		~SF	非负数
jg Label	jnle	~(SF ^ OF) & ~ZF	大于 (有符号>)
jge Label	jnl	~(SF ^ OF)	大于或等于 (有符号>=)
jl Label	jnge	SF ^ OF	小于 (有符号<)
jle Label	jng	(SF ^ OF) ZF	小于或等于 (有符号<=)
ja Label	jnbe	~CF & ~ZF	超过 (无符号>)
jae Label	jnb	~CF	超过或相等 (无符号>=)
jb Label	jnae	CF	低于 (无符号<)
jbe Label	jna	CF ZF	低于或相等 (无符号<=)

cmp: compare 的缩写，比较指令 rdi - rsi

jge: jump great equal 的缩写，大于等于跳转指令

```
absdiff_se:
    → cmp     rdi, rsi
    jge      .L2
    add      QWORD PTR lt_cnt[rip], 1
    mov      rax, rsi
    sub      rax, rdi
    ret
```

```
    add      QWORD PTR ge_cnt[rip], 1
    mov      rax, rdi
    sub      rax, rsi
    ret
```

X86-64 指令系统：选择语句 (if...else...)

00000000004004ed <absdiff_se>:

4004ed:	48 39 f7	cmp	rdi,rsi
4004f0:	7d 0f	jge	400501 <absdiff_se+0x14>
4004f2:	48 83 05 46 0b 20 00	add	QWORD PTR [rip+0x200b46],0x1
4004f9:	01		
4004fa:	48 89 f0	mov	rax,rsi
4004fd:	48 29 f8	sub	rax,rdi
400500:	c3	ret	
400501:	48 83 05 2f 0b 20 00	add	QWORD PTR [rip+0x200b2f],0x1
400508:	01		
400509:	48 89 f8	mov	rax,rdi
40050c:	48 29 f0	sub	rax,rsi
40050f:	c3	ret	

X86-64 指令系统：选择语句 (if...else...)

```
long lt_cnt = 0;
long ge_cnt = 0;
long absdiff_se(long x, long v) {
    %rdi      %rsi
    ↑        ↑
    long x    long v
```

cmp: compare 的缩写，比较指令

jge: jump if great equal 的缩写，大于等于跳转指令

```
absdiff_se:
    cmp    rdi, rsi
    jge    .L2
    add    QWORD PTR lt_cnt[rip], 1
    mov    rax, rsi
    sub    rax, rdi
    ret
```

指令	同义名	跳转条件	描述
jmp Label		1	直接跳转
jmp *Operand		1	间接跳转
je Label	jz	ZF	相等/零
jne Label	jnz	~ZF	不相等/非零
js Label		SF	负数
jns Label		~SF	非负数
jg Label	jnle	~(SF ^ OF) & ~ZF	大于 (有符号>)
jge Label	jnl	~(SF ^ OF)	大于或等于 (有符号>=)
jl Label	jnge	SF ^ OF	小于 (有符号<)
jle Label	jng	(SF ^ OF) ZF	小于或等于 (有符号<=)
ja Label	jnbe	~CF & ~ZF	超过 (无符号>)
jae Label	jnb	~CF	超过或相等 (无符号>=)
jb Label	jnae	CF	低于 (无符号<)
jbe Label	jna	CF ZF	低于或相等 (无符号<=)

```
add    QWORD PTR ge_cnt[rip], 1
mov    rax, rdi
sub    rax, rsi
ret
```

```
long lt_cnt = 0;
```

```
long ge_cnt = 0;
```

```
long absdiff_se(long x, long y) {
```

```
    long result;
```

```
    if (x < y) {
```

```
        lt_cnt++;
```

```
        result = y - x;
```

```
    } else {
```

```
        g00000000004004ed <absdiff_se>:
```

```
        4004ed:      48 39 f7
```

```
        4004f0:      7d 0f
```

```
    }  
    4004f2:      48 83 05 46 0b 20 00
```

```
    return  
    4004f9:      01
```

```
    4004fa:      48 89 f0
```

```
    4004fd:      48 29 f8
```

```
    400500:      c3
```

```
    400501:      48 83 05 2f 0b 20 00
```

```
    400508:      01
```

```
    400509:      48 89 f8
```

```
    40050c:      48 29 f0
```

```
    40050f:      c3
```

```
    cmp     rdi,rsi
```

```
    jge     400501 <absdiff_se+0x14>
```

```
    add     QWORD PTR [rip+0x200b46],0x1
```

```
    mov     rax,rsi
```

```
    sub     rax,rdi
```

```
    ret
```

```
    add     QWORD PTR [rip+0x200b2f],0x1
```

```
    mov     rax,rdi
```

```
    sub     rax,rsi
```

```
    ret
```

0x0000099cca02

函数栈
(运行时创建)

← %rsp (栈指针)



运行时堆
(由 malloc 创建)

← brk

0x000000811a02

读/写数据段
(.data、.bss)

0x000000600a00

只读代码段
(.init、.text、.rodata)

从可执行文件中加载

0x000000400000

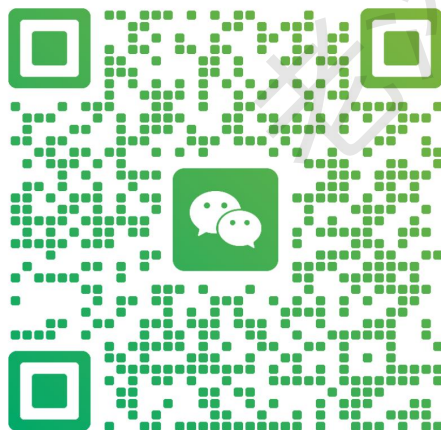
0x000000000000

X86-64 指令系统：选择语句 (if...else...)

```
void cond(long rdia, long rsi*p) {  
    if (p && a > *p)  
        *p = a;  
}
```

加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】

p != NULL



扫一扫上面的二维码图案，加我为朋友。

cond:

```
→ test    rsi, rsi  
   je      .L1  
   cmp     QWORD PTR [rsi], rdi  
   jge     .L1  
   mov     QWORD PTR [rsi], rdi
```

.L1:

```
ret
```

test 指令

TEST reg/mem, reg : 对寄存器或内存操作数与寄存器操作数进行**按位与**运算

TEST reg/mem, imm: 对寄存器或内存操作数与立即数进行**按位与**运算

标志寄存器

.....	PF	ZF	SF	CF	OF
-------	----	----	----	----	----

零标志位 (ZF) : 如果按位与的结果为 0, 则 ZF 被置为 1; 否则, ZF 被置为 0

符号标志位 (SF) : 根据按位与结果的最高位设置, 若最高位为 1, 则 SF 为 1; 否则, SF 为 0

奇偶标志位 (PF) : 若按位与结果中 1 的个数为偶数, 则 PF 为 1; 否则, PF 为 0

进位标志位 (CF) 和溢出标志位 (OF) : 总是被置为 0。

X86-64 指令系统：选择语句 (if...else...) je: jump if equal 的缩写，等于零就跳转指令

```
void cond(long a, long *p) {
    if (p && a > *p)
        *p = a;
}
```



指令	同义名	跳转条件	描述
jmp Label		1	直接跳转
jmp *Operand		1	间接跳转
je Label	jz	ZF	相等/零
jne Label	jnz	~ZF	不相等/非零
js Label		SF	负数
jns Label		~SF	非负数
jg Label	jnle	~(SF ^ OF) & ~ZF	大于 (有符号>)
jge Label	jnl	~(SF ^ OF)	大于或等于 (有符号>=)
jl Label	jnge	SF ^ OF	小于 (有符号<)
jle Label	jng	(SF ^ OF) ZF	小于或等于 (有符号<=)
ja Label	jnbe	~CF & ~ZF	超过 (无符号>)
jae Label	jnb	~CF	超过或相等 (无符号>=)
jb Label	jnae	CF	低于 (无符号<)
jbe Label	jna	CF ZF	低于或相等 (无符号<=)

```
cond:
    → test    rsi, rsi
      je      .L1
      cmp     QWORD PTR [rsi], rdi
      jge     .L1
      mov     QWORD PTR [rsi], rdi
.L1:
      ret
```

X86-64 指令系统：循环语句 (do...while)

jg: jump if great 的缩写，大于就跳转指令

```
long fact_do(long n) {  
    → long result = 1;  
    do {  
        ↑  
        rdi
```

```
fact_do:  
    → mov     eax, 1  
.L5:  
    imul     rax, rdi  
    sub      rdi, 1  
    cmp      rdi, 1  
    jg       .L5  
    ret
```

指令	同义名	跳转条件	描述
jmp Label		1	直接跳转
jmp *Operand		1	间接跳转
je Label	jz	ZF	相等/零
jne Label	jnz	~ZF	不相等/非零
js Label		SF	负数
jns Label		~SF	非负数
jg Label	jnle	~(SF ^ OF) & ~ZF	大于 (有符号>)
jge Label	jnl	~(SF ^ OF)	大于或等于 (有符号>=)
jl Label	jnge	SF ^ OF	小于 (有符号<)
jle Label	jng	(SF ^ OF) ZF	小于或等于 (有符号<=)
ja Label	jnbe	~CF & ~ZF	超过 (无符号>)
jae Label	jnb	~CF	超过或相等 (无符号>=)
jb Label	jnae	CF	低于 (无符号<)
jbe Label	jna	CF ZF	低于或相等 (无符号<=)

X86-64 指令系统：循环语句 (do...while)

jg: jump if great 的缩写，大于就跳转指令

```
                                %rdi
                                ↑
long fact_do(long n) {
    long result = 1;
    do {
        result *= n;
        n = n - 1;
    } while (n > 1);
    return result;
}
```



```
fact_do:
    mov     eax, 1
.L5:
    imul    rax, rdi
    sub     rdi, 1
    cmp     rdi, 1
    → jg     .L5
    ret
```



```
loop:
    body-statement
    t = test-expr;
    if (t)
        goto loop;
```

X86-64 指令系统：循环语句 (while)

```
long fact_while(long n) {  
    → long result = 1;  
    while (n > 1) {  
        result *= n;  
        n = n - 1;  
    }  
    return result;  
}
```

rdi
↑

优化级别: -Og

```
fact_while:  
    → mov     eax, 1  
    jmp      .L2  
.L3:  
    → imul    rax, rdi  
    sub      rdi, 1  
.L2:  
    → cmp     rdi, 1  
    jg       .L3  
    ret
```



```
    goto test;  
loop:  
    body-statement  
test:  
    t = test-expr;  
    if (t)  
        goto loop;
```

优化级别: -O1

```
long fact while(long n) {  
    ↑  
    rdi
```

fact_while:

```
→ cmp    rdi, 1  
    .L4  
    eax, 1
```

指令	同义名	跳转条件	描述
jmp Label		1	直接跳转
jmp *Operand		1	间接跳转
je Label	jz	ZF	相等/零
jne Label	jnz	~ZF	不相等/非零
js Label		SF	负数
jns Label		~SF	非负数
jg Label	jnle	~(SF ^ OF) & ~ZF	大于 (有符号>)
jge Label	jnl	~(SF ^ OF)	大于或等于 (有符号>=)
jl Label	jnge	SF ^ OF	小于 (有符号<)
jle Label	jng	(SF ^ OF) ZF	小于或等于 (有符号<=)
ja Label	jnbe	~CF & ~ZF	超过 (无符号>)
jae Label	jnb	~CF	超过或相等 (无符号>=)
jb Label	jnae	CF	低于 (无符号<)
jbe Label	jna	CF ZF	低于或相等 (无符号<=)

```
rax, rdi  
rdi, 1  
rdi, 1  
.L3
```

```
eax, 1
```

X86-64 指令系统：循环语句 (while)

jle: jump less equal 的缩写，小于等于就跳转指令
jne: jump not equal 的缩写，不等于就跳转指令

优化级别: -O1

```
long fact_while(long n) {  
    ↑  
    %rdi  
    while(n) {
```

fact_while:

```
    cmp    rdi, 1  
    .L4:  
    mov    eax, 1
```

```
    mov    rax, rdi
```

```
    mov    rdi, 1
```

```
    mov    rdi, 1
```

.L3:

```
    mov    eax, 1
```

指令	同义名	跳转条件	描述
jmp Label		1	直接跳转
jmp *Operand		1	间接跳转
je Label	jz	ZF	相等/零
jne Label	jnz	~ZF	不相等/非零
js Label		SF	负数
jns Label		~SF	非负数
jg Label	jnle	~(SF ^ OF) & ~ZF	大于 (有符号>)
jge Label	jnl	~(SF ^ OF)	大于或等于 (有符号>=)
jl Label	jnge	SF ^ OF	小于 (有符号<)
jle Label	jng	(SF ^ OF) ZF	小于或等于 (有符号<=)
ja Label	jnb	~CF & ~ZF	超过 (无符号>)
jae Label	jnb	~CF	超过或相等 (无符号>=)
jb Label	jnae	CF	低于 (无符号<)
jbe Label	jna	CF ZF	低于或相等 (无符号<=)

X86-64 指令系统：循环语句 (while)

jle: jump less equal 的缩写，小于等于就跳转指令

jne: jump not equal 的缩写，不等于就跳转指令

优化级别: -O1

```
long fact_while(long n) {  
    long result = 1;  
    → while (n > 1) {  
        result *= n;  
        n = n - 1;  
    }  
    return result;  
}
```

rdi
↑

```
fact_while:  
    cmp    rdi, 1  
    jle    .L4  
    mov    eax, 1  
.L3:  
    imul   rax, rdi  
    sub    rdi, 1  
    cmp    rdi, 1  
    → jne  .L3  
    ret  
.L4:  
    mov    eax, 1  
    ret
```



```
t = test-expr;  
if (!t)  
    goto done;  
loop:  
    body-statement  
    t = test-expr;  
    if (t)  
        goto loop;  
done:
```

X86-64 指令系统：循环语句 (for)

```
long fact_for(long n) {  
    long result = 1;  
    long i;  
    for (i = 2; i <= n; i++)  
        result *= i;  
    return result;  
}
```



```
long fact_while(long n) {  
    long result = 1;  
    while (n > 1) {  
        result *= n;  
        n = n - 1;  
    }  
    return result;  
}
```

```
for (init-expr; test-expr; update-expr)  
    body-statement
```



```
init-expr;  
while (test-expr) {  
    body-statement  
    update-expr;  
}
```

X86-64 指令系统：循环语句 (for)

```
long fact_for(long n) {  
    long result = 1;  
    long i;  
    for (i = 2; i <= n; i++)  
        result *= i;  
    return result;  
}
```



```
long fact_while(long n) {  
    long result = 1;  
    while (n > 1) {  
        result *= n;  
        n = n - 1;  
    }  
    return result;  
}
```

优化级别: -Og

fact_for:

mov eax, 2

mov edx, 1

jmp .L2

.L3:

imul rdx, rax

add rax, 1

.L2:

cmp rax, rdi

jle .L3

mov rax, rdx

ret

```
    goto test;  
loop:  
    body-statement  
test:  
    t = test-expr;  
    if (t)  
        goto loop;
```

X86-64 指令系统: 循环语句 (for)

```
int sum_arr(int arr[], int n) {
    int res = 0;
    for (int i = 0; i < n; i++) {
        res += arr[i];
    }
    return res;
}
```



优化级别: -O1

```
t = test-expr;
if (!t)
    goto done;
loop:
    body-statement
    t = test-expr;
    if (t)
        goto loop;
done:
```

00000000004004ed <sum_arr>:

→ 4004ed:	85 f6	test	esi,esi
4004ef:	7e 17	jle	400508 <sum_arr+0x1b>
4004f1:	ba 00 00 00 00	mov	edx,0x0
4004f6:	b8 00 00 00 00	mov	eax,0x0
4004fb:	03 04 97	add	eax,DWORD PTR [rdi+rdx*4]
4004fe:	48 83 c2 01	add	rdx,0x1
400502:	39 d6	cmp	esi,edx
400504:	7f f5	jg	4004fb <sum_arr+0xe>
400506:	f3 c3	repz ret	
400508:	b8 00 00 00 00	mov	eax,0x0
40050d:	c3	ret	

test 指令

TEST reg/mem, reg : 对寄存器或内存操作数与寄存器操作数进行**按位与**运算

TEST reg/mem, imm: 对寄存器或内存操作数与立即数进行**按位与**运算

标志寄存器

.....	PF	ZF	SF	CF	OF
-------	----	----	----	----	----

零标志位 (ZF) : 如果按位与的结果为 0, 则 ZF 被置为 1; 否则, ZF 被置为 0

符号标志位 (SF) : 根据按位与结果的最高位设置, 若最高位为 1, 则 SF 为 1; 否则, SF 为 0

奇偶标志位 (PF) : 若按位与结果中 1 的个数为偶数, 则 PF 为 1; 否则, PF 为 0

进位标志位 (CF) 和溢出标志位 (OF) : 总是被置为 0。

X86-64 指令系统：循环语句 (for)

```
int sum_arr(int arr[], int n) {
    int res = 0;
    for (int i = 0; i < n; i++) {
        res += arr[i];
    }
    return res;
}
```

jne Label	jnz	~ZF	不相等/非零
js Label		SF	负数
jns Label		~SF	非负数
jg Label	jnle	~(SF ^ OF) & ~ZF	大于 (有符号>)
jge Label	jnl	~(SF ^ OF)	大于或等于 (有符号>=)
jl Label	jnge	SF ^ OF	小于 (有符号<)
jle Label	jng	(SF ^ OF) ZF	小于或等于 (有符号<=)
ja Label	jnbe	~CF & ~ZF	超过 (无符号>)
jae Label	jnb	~CF	超过或相等 (无符号>=)
jb Label	jnae	CF	低于 (无符号<)
jbe Label	jna	CF ZF	低于或相等 (无符号<=)

```
00000000004004ed <sum_arr>:
-> 4004ed:      85 f6      test    esi,esi
    4004ef:      7e 17      jle     400508 <sum_arr+0x1b>
    4004f1:      ba 00 00 00 00      mov     edx,0x0
    4004f6:      b8 00 00 00 00      mov     eax,0x0
    4004fb:      03 04 97      add     eax,DWORD PTR [rdi+rdx*4]
    4004fe:      48 83 c2 01      add     rdx,0x1
    400502:      39 d6      cmp     esi,edx
    400504:      7f f5      jg      4004fb <sum_arr+0xe>
    400506:      f3 c3      repz   ret
-> 400508:      b8 00 00 00 00      mov     eax,0x0
    40050d:      c3      ret
```

X86-64 指令系统: 循环语句 (for)

```
int sum_arr(int arr[], int n) {
    int res = 0;
    → for (int i = 0; i < n; i++) {
        res += arr[i];
    }
    return res;
}
```



优化级别: -O1

```
t = test-expr;
if (!t)
    goto done;
loop:
    body-statement
    t = test-expr;
    if (t)
        goto loop;
done:
```

```
000000000004004ed <sum_arr>:
4004ed: 85 f6      test    esi,esi
→ 4004ef: 7e 17      jle     400508 <sum_arr+0x1b>
4004f1: ba 00 00 00 00  mov    edx,0x0 初始化 i = 0
4004f6: b8 00 00 00 00  mov    eax,0x0 初始化 res = 0
4004fb: 03 04 97     add    eax,DWORD PTR [rdi+rdx*4]
4004fe: 48 83 c2 01   add    rdx,0x1  i++
400502: 39 d6      cmp    esi,edx
400504: 7f f5      jg     4004fb <sum_arr+0xe>
400506: f3 c3      repz   ret
400508: b8 00 00 00 00  mov    eax,0x0
40050d: c3         ret
```

X86-64 指令系统：过程调用的机器级表示

func.c

编译：gcc -o func func.c

查看 intel 格式的汇编代码：objdump -d -M intel func

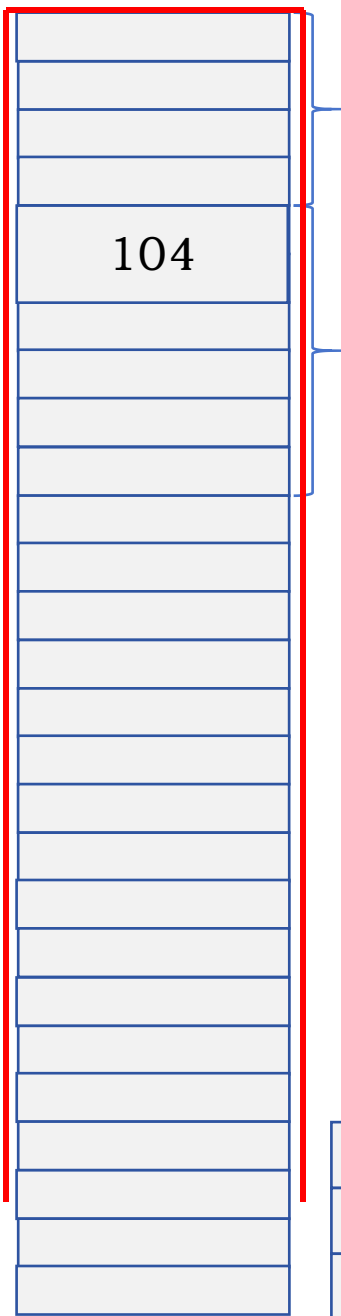
```
int _sub(int a, int b) {  
    return a - b;  
}
```

```
int _add(int a, int b) {  
    int res = _sub(a, b);  
    return res + b;  
}
```

```
int main() {  
    int a = 3;  
    int b = 2;  
    int c = _add(a, b);  
    return 0;  
}
```

主存储器

rbp→104
100
96
rsp→92
88
84
80
76
72
68
64
60
56
52
48
44
40
36
32
28
24
20
16
12
8
4
0

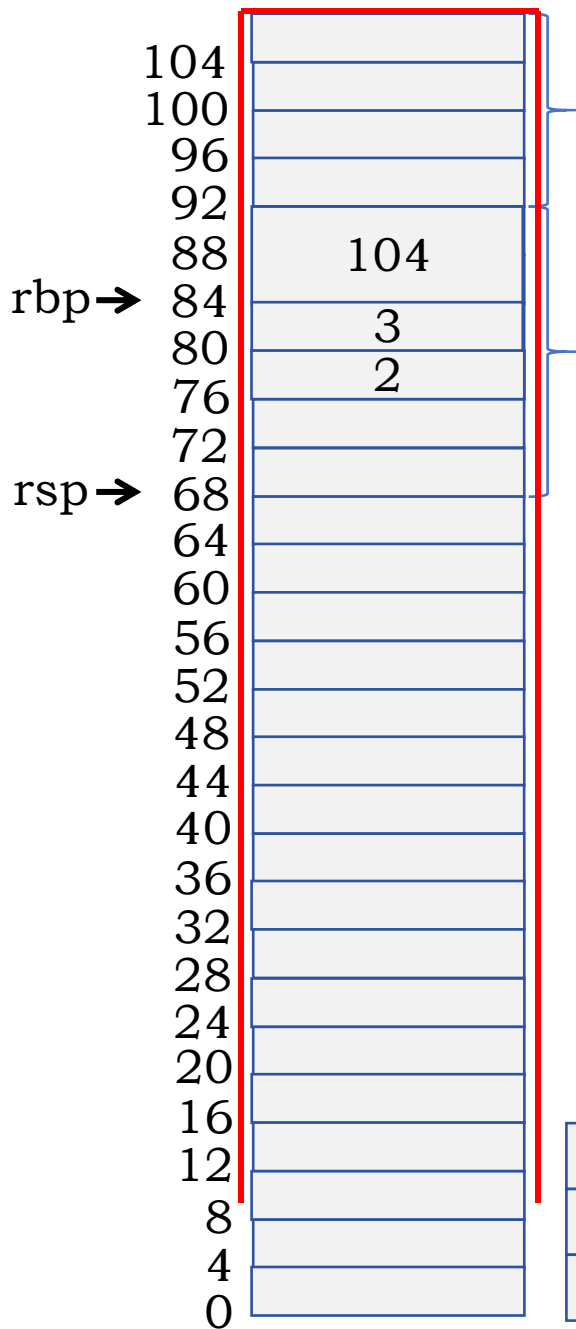


000000000040052d <main>:

```
→ 40052d: 55          push    rbp
40052e: 48 89 e5     mov     rbp, rsp
400531: 48 83 ec 10   sub     rsp, 0x10
400535: c7 45 fc 03 00 00 00 mov     DWORD PTR [rbp-0x4], 0x3
40053c: c7 45 f8 02 00 00 00 mov     DWORD PTR [rbp-0x8], 0x2
400543: 8b 55 f8     mov     edx, DWORD PTR [rbp-0x8]
400546: 8b 45 fc     mov     eax, DWORD PTR [rbp-0x4]
400549: 89 d6       mov     esi, edx
40054b: 89 c7       mov     edi, eax
40054d: e8 b1 ff ff ff call    400503 <_add>
400552: 89 45 f4     mov     DWORD PTR [rbp-0xc], eax
400555: b8 00 00 00 00 mov     eax, 0x0
40055a: c9         leave
40055b: c3         ret
```

PC	0x40053c
rbp	84
rsp	68

主存储器



000000000040052d <main>:

```

40052d: 55
40052e: 48 89 e5
400531: 48 83 ec 10
400535: c7 45 fc 03 00 00 00
40053c: c7 45 f8 02 00 00 00
400543: 8b 55 f8
400546: 8b 45 fc
400549: 89 d6
40054b: 89 c7
40054d: e8 b1 ff ff ff
400552: 89 45 f4
400555: b8 00 00 00 00
40055a: c9
40055b: c3
    
```

```

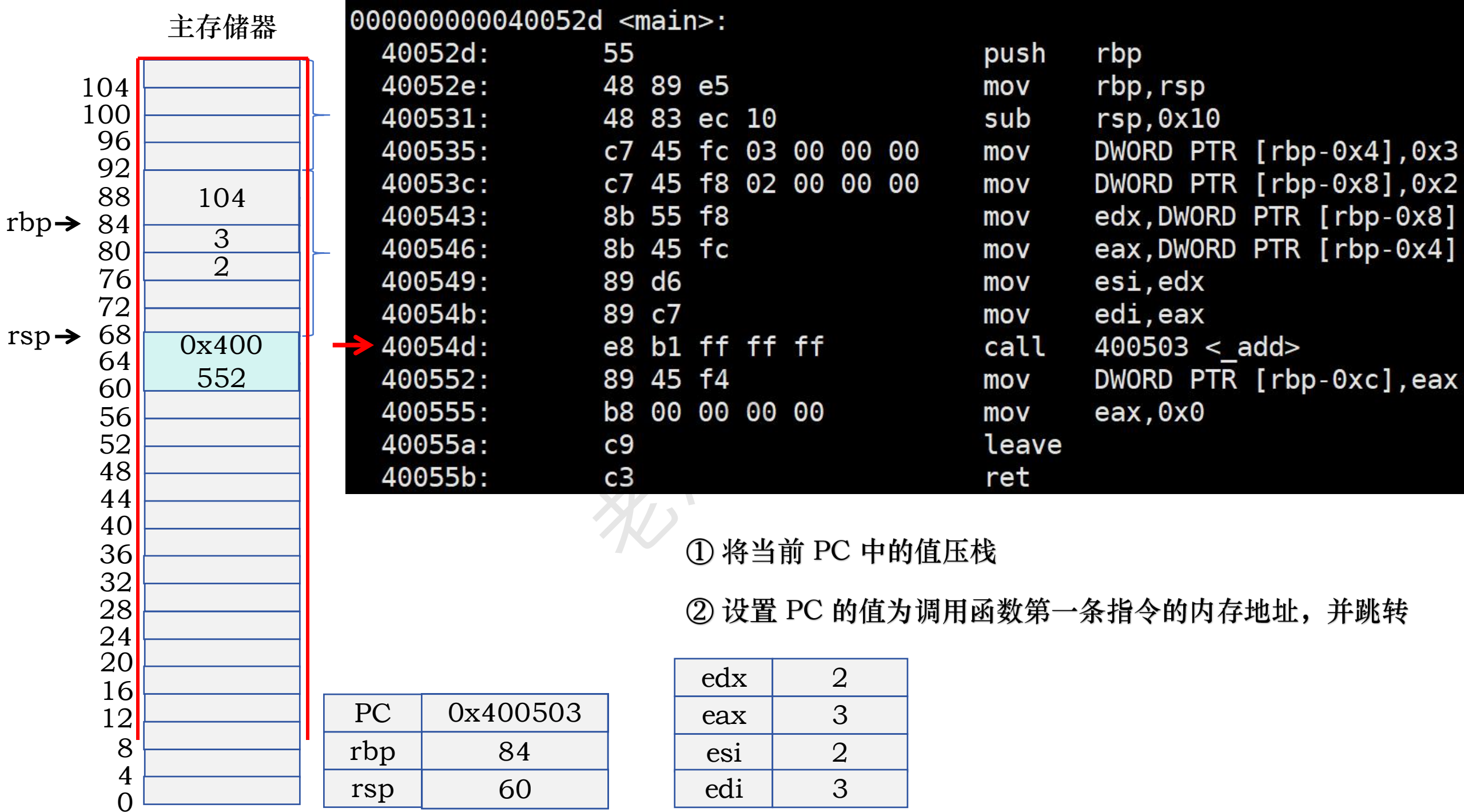
push    rbp
mov     rbp, rsp
sub     rsp, 0x10
mov     DWORD PTR [rbp-0x4], 0x3
mov     DWORD PTR [rbp-0x8], 0x2
mov     edx, DWORD PTR [rbp-0x8]
mov     eax, DWORD PTR [rbp-0x4]
mov     esi, edx
mov     edi, eax
call    400503 <_add>
mov     DWORD PTR [rbp-0xc], eax
mov     eax, 0x0
leave
ret
    
```

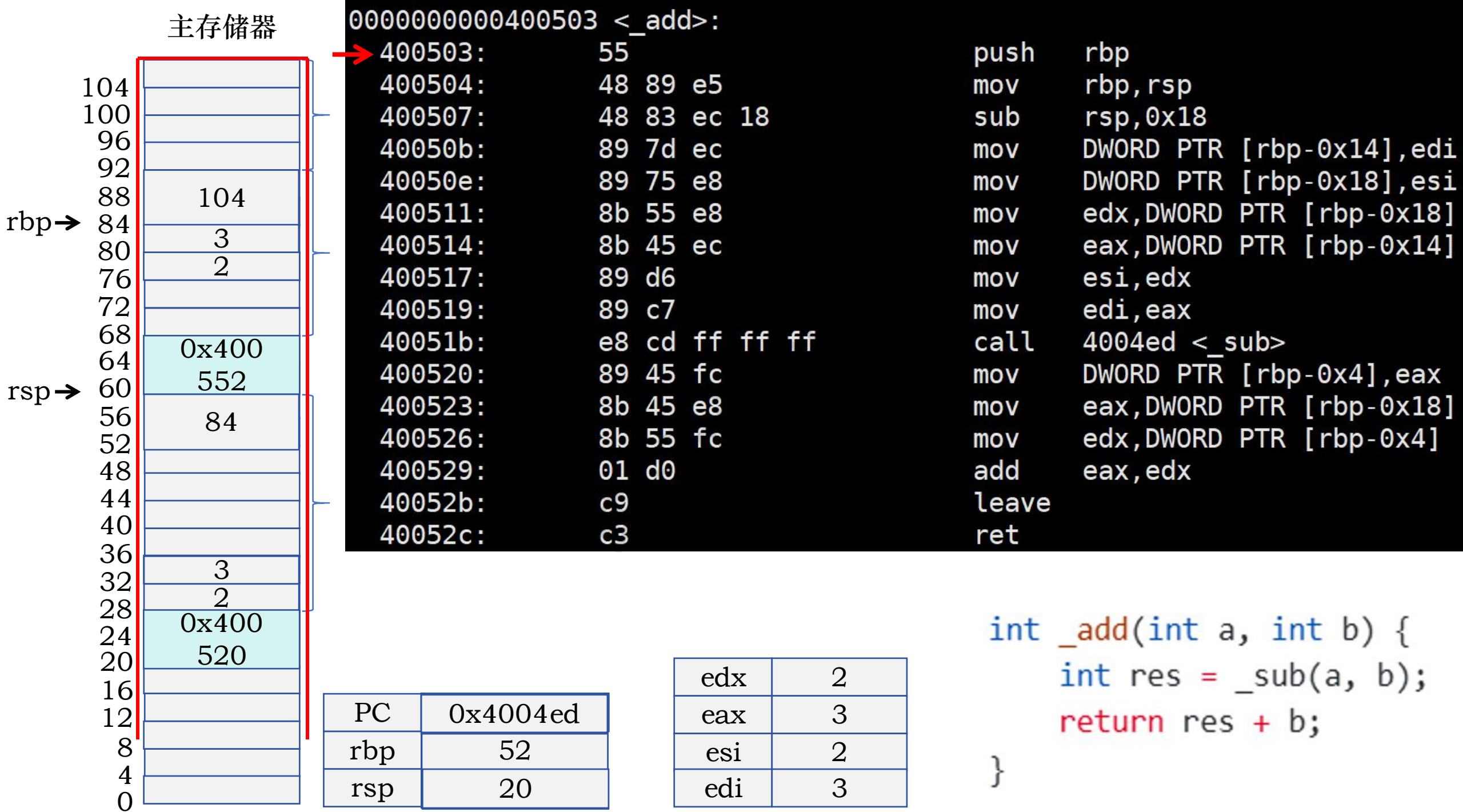
PC	0x400552
rbp	84
rsp	68

edx	2
eax	3
esi	2
edi	3

```

int main() {
    int a = 3;
    int b = 2;
    int c = _add(a, b);
    return 0;
}
    
```





主存储器

00000000004004ed <_sub>:

```

→ 4004ed:      55      push    rbp
4004ee:      48 89 e5     mov     rbp, rsp
4004f1:      89 7d fc     mov     DWORD PTR [rbp-0x4], edi
4004f4:      89 75 f8     mov     DWORD PTR [rbp-0x8], esi
4004f7:      8b 45 f8     mov     eax, DWORD PTR [rbp-0x8]
4004fa:      8b 55 fc     mov     edx, DWORD PTR [rbp-0x4]
4004fd:      29 c2       sub     edx, eax
4004ff:      89 d0       mov     eax, edx
400501:      5d       pop     rbp
400502:      c3       ret
    
```

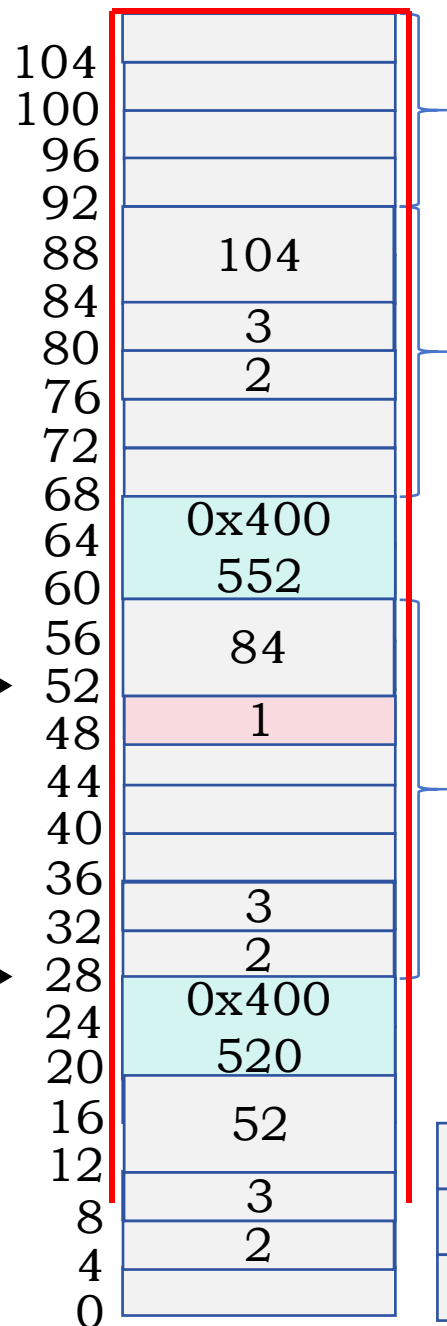
① 将当栈顶元素出栈，保存到 PC

② 跳转到 PC 指向的指令

PC	0x400520
rbp	52
rsp	28

edx	1
eax	1
esi	2
edi	3

主存储器



0000000000400503 <_add>:

```

400503: 55
400504: 48 89 e5
400507: 48 83 ec 18
40050b: 89 7d ec
40050e: 89 75 e8
400511: 8b 55 e8
400514: 8b 45 ec
400517: 89 d6
400519: 89 c7
40051b: e8 cd ff ff ff
400520: 89 45 fc
400523: 8b 45 e8
400526: 8b 55 fc
400529: 01 d0
40052b: c9
40052c: c3
    
```

```

push    rbp
mov     rbp, rsp
sub     rsp, 0x18
mov     DWORD PTR [rbp-0x14], edi
mov     DWORD PTR [rbp-0x18], esi
mov     edx, DWORD PTR [rbp-0x18]
mov     eax, DWORD PTR [rbp-0x14]
mov     esi, edx
mov     edi, eax
call    4004ed <_sub>
mov     DWORD PTR [rbp-0x4], eax
mov     eax, DWORD PTR [rbp-0x18]
mov     edx, DWORD PTR [rbp-0x4]
add     eax, edx
leave
ret
    
```

① mov rsp,rbp

② pop rbp

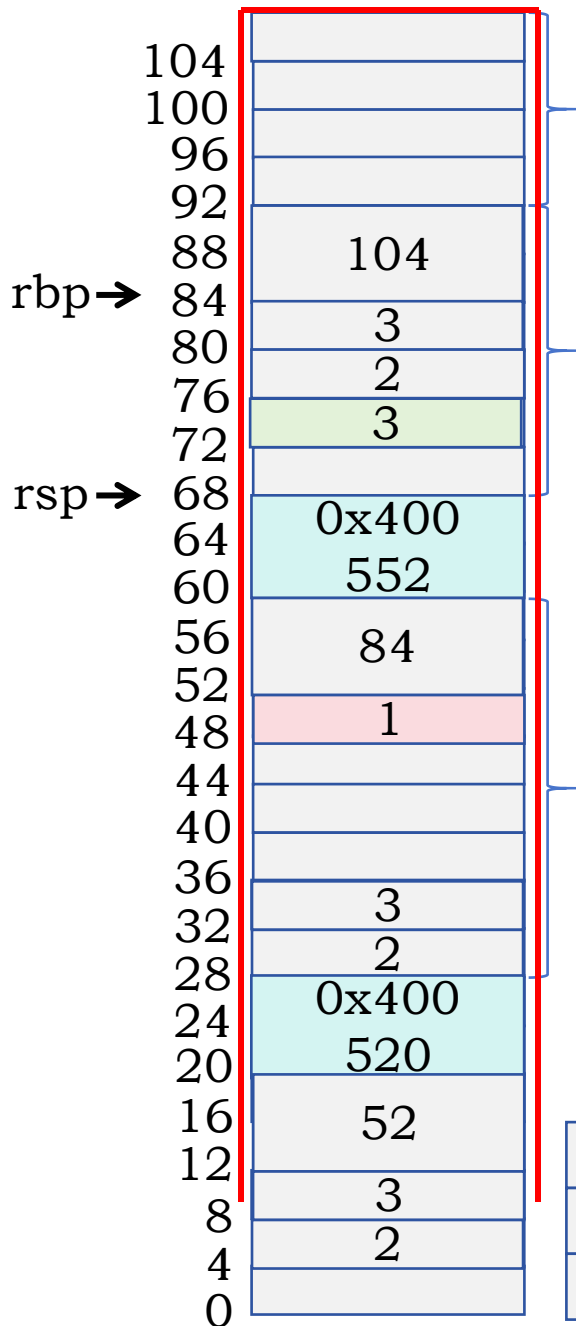
PC	0x400552
rbp	84
rsp	68

edx	1
eax	3
esi	2
edi	3

```

int _add(int a, int b) {
    int res = _sub(a, b);
    return res + b;
}
    
```

主存储器



000000000040052d <main>:

```

40052d: 55
40052e: 48 89 e5
400531: 48 83 ec 10
400535: c7 45 fc 03 00 00 00
40053c: c7 45 f8 02 00 00 00
400543: 8b 55 f8
400546: 8b 45 fc
400549: 89 d6
40054b: 89 c7
40054d: e8 b1 ff ff ff
400552: 89 45 f4
400555: b8 00 00 00 00
40055a: c9
40055b: c3

```

```

push    rbp
mov     rbp, rsp
sub     rsp, 0x10
mov     DWORD PTR [rbp-0x4], 0x3
mov     DWORD PTR [rbp-0x8], 0x2
mov     edx, DWORD PTR [rbp-0x8]
mov     eax, DWORD PTR [rbp-0x4]
mov     esi, edx
mov     edi, eax
call    400503 <_add>
mov     DWORD PTR [rbp-0xc], eax
mov     eax, 0x0
leave
ret

```

① mov rsp,rbp

② pop rbp

```

int main() {
    int a = 3;
    int b = 2;
    int c = _add(a, b);
    return 0;
}

```

PC	0x40055b
rbp	104
rsp	84

edx	1
eax	0
esi	2
edi	3

X86-64 指令系统：过程调用的机器级表示

```
#include <stdio.h>

int _sub(int a, int b) {
    return a - b;
}

int _add(int a, int b) {
    int res = _sub(a, b);
    return res + b;
}

int main() {
    int a = 3;
    int b = 2;
    int c = _add(a, b);
    printf("c = %d\n", c);
    return 0;
}
```

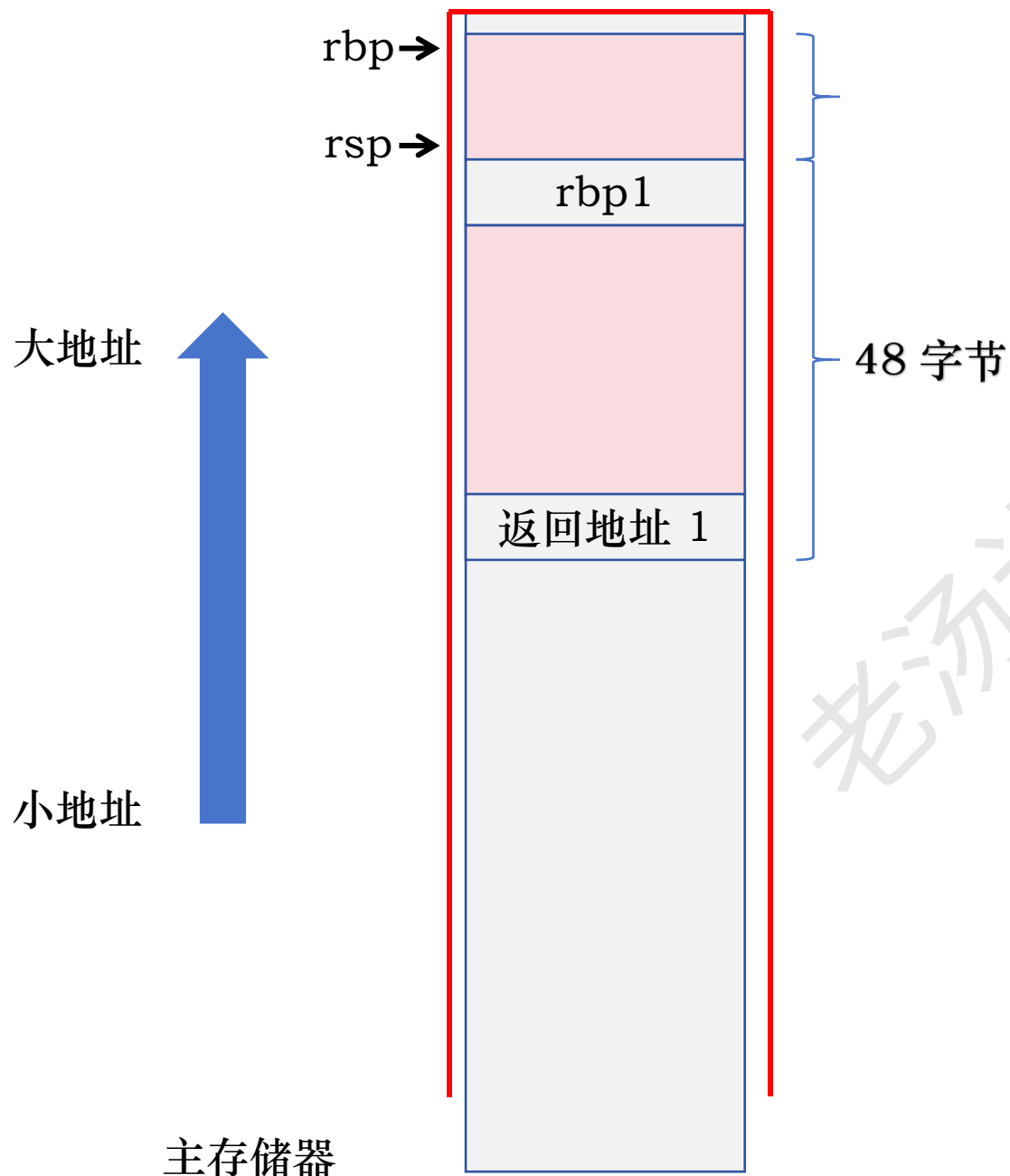


```
main:
    push rbp;
    mov rbp, rsp;
    sub rsp, 0x20;
    ...
    call _add
    ...
    leave;
    ret

_add:
    push rbp;
    mov rbp, rsp;
    sub rsp, 0x18;
    ...
    call _sub
    ...
    leave;
    ret

_sub:
    push rbp;
    mov rbp, rsp;
    ...
    pop rbp;
    ret
```

X86-64 指令系统：过程调用的机器级表示



main:

```
→ push rbp;  
   mov rbp rsp;  
   sub rsp 0x20;
```

保存上一个栈帧的顶部内存地址
rbp 指向当前栈帧的顶部位置
分配 32 字节栈内存

```
...  
call _add
```

```
...  
leave;  
ret
```

**加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】**

_add:

```
→ push rbp;  
   mov rbp rsp;  
   sub rsp 0x18;
```

```
...  
call _sub
```

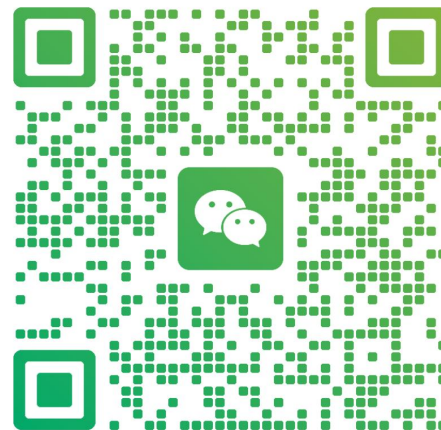
```
...  
leave;  
ret
```

_sub:

```
push rbp;  
mov rbp rsp;  
...  
pop rbp;  
ret
```



老汤
浙江 杭州

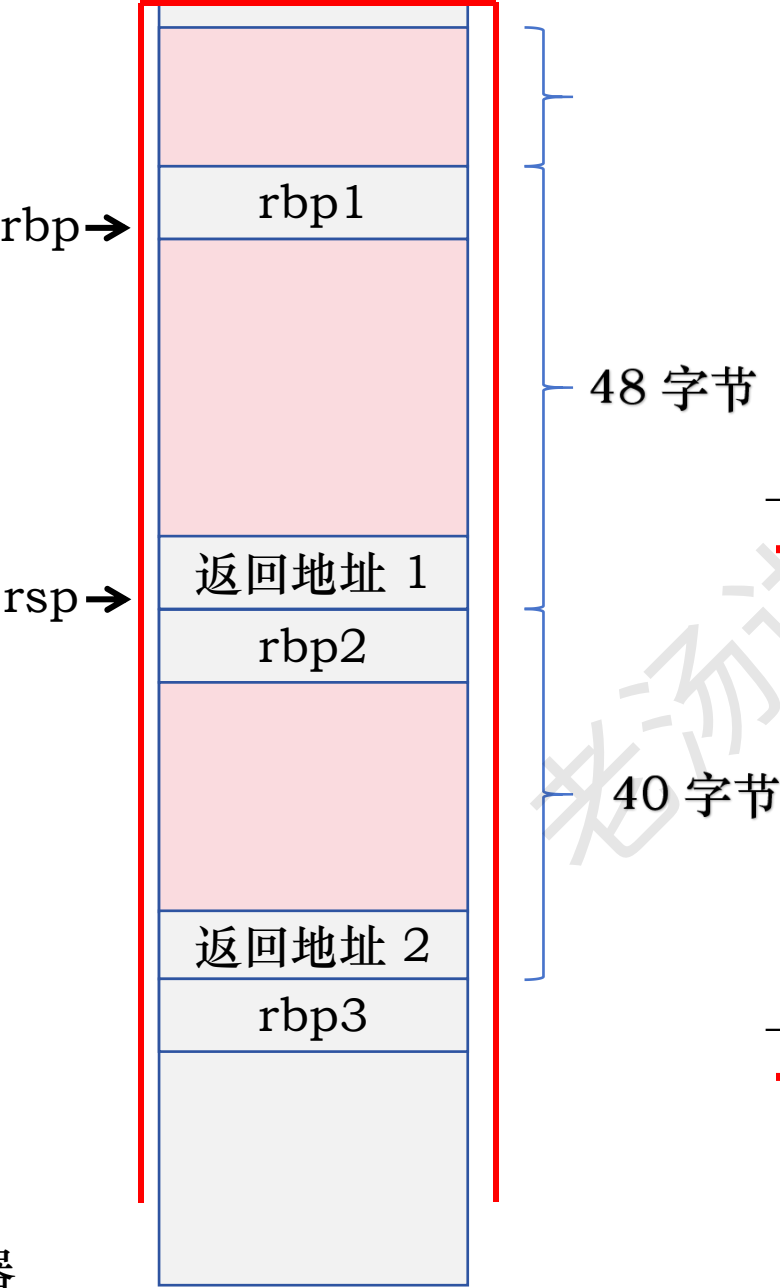


扫一扫上面的二维码图案，加我为朋友。

X86-64 指令系统：过程调用的机器级表示



主存储器



main:

```
push rbp;
mov rbp rsp;
sub rsp 0x20;
...
call _add
...
leave;
ret
```

保存上一个栈帧的顶部内存地址
rbp 指向当前栈帧的顶部位置
分配 32 字节栈内存

→ _add:

```
push rbp;
mov rbp rsp;
sub rsp 0x18;
...
call _sub
...
leave;
ret
```

保存上一个栈帧的顶部内存地址
rbp 指向当前栈帧的顶部位置
分配 24 字节栈内存

→ _sub:

```
push rbp;
mov rbp rsp;
...
pop rbp;
ret
```

保存上一个栈帧的顶部内存地址
rbp 指向当前栈帧的顶部位置

X86-64 指令系统：过程调用的机器级表示



主存储器

rbp →



48 字节

40 字节

main:

```
push rbp;  
mov rbp rsp;  
sub rsp 0x20;  
...  
call _add  
...  
leave;  
ret
```

保存上一个栈帧的顶部内存地址
rbp 指向当前栈帧的顶部位置
分配 32 字节栈内存

_add:

```
push rbp;  
mov rbp rsp;  
sub rsp 0x18;  
...  
call _sub  
→ ...  
leave;  
ret
```

保存上一个栈帧的顶部内存地址
rbp 指向当前栈帧的顶部位置
分配 24 字节栈内存

_sub:

```
push rbp;  
→ mov rbp rsp;  
...  
pop rbp;  
ret
```

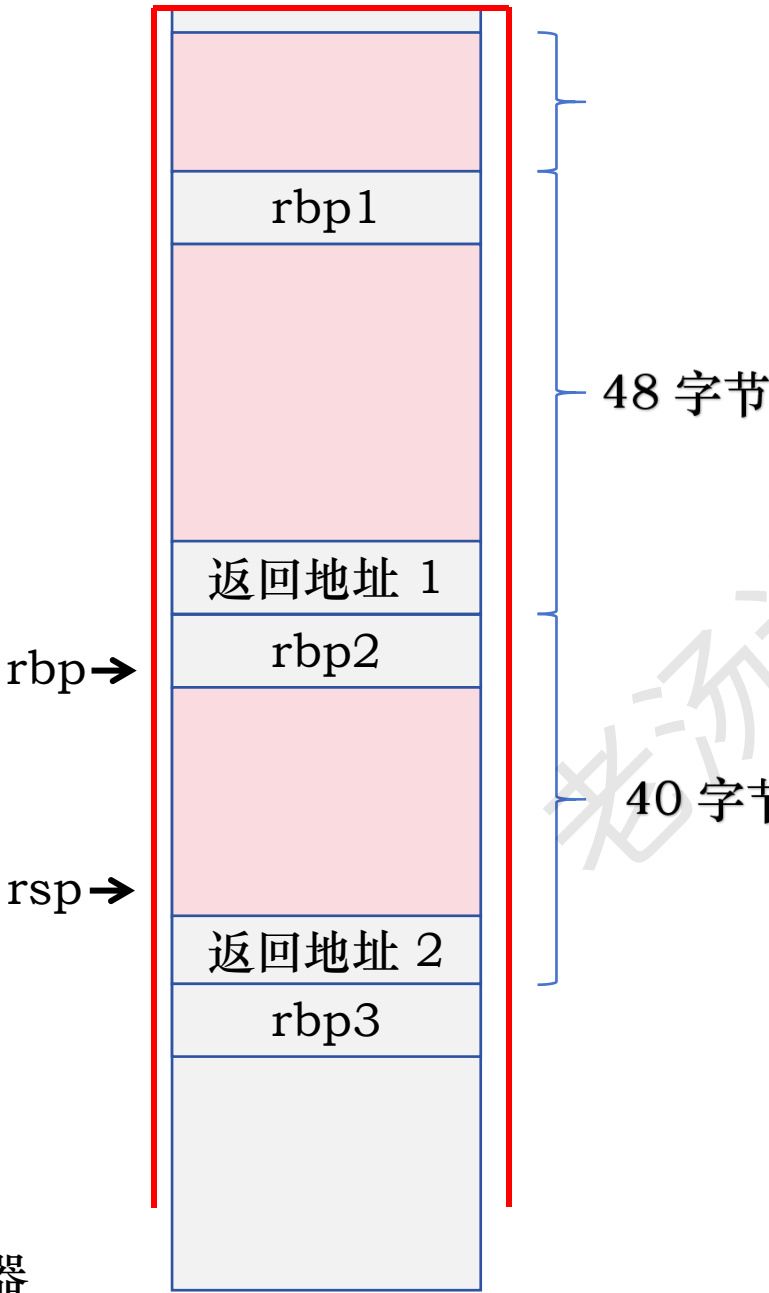
保存上一个栈帧的顶部内存地址
rbp 指向当前栈帧的顶部位置
rbp 恢复到上一个栈帧顶部

X86-64 指令系统：过程调用的机器级表示

大地址

小地址

主存储器



main:

```
push rbp;  
mov rbp rsp;  
sub rsp 0x20;
```

保存上一个栈帧的顶部内存地址
rbp 指向当前栈帧的顶部位置
分配 32 字节栈内存

```
...  
call _add
```

```
→ ...  
leave;  
ret
```

① mov rsp,rbp; ② pop rbp

_add:

```
push rbp;  
mov rbp rsp;  
sub rsp 0x18;
```

保存上一个栈帧的顶部内存地址
rbp 指向当前栈帧的顶部位置
分配 24 字节栈内存

```
...  
call _sub
```

```
→ ...  
leave;  
ret
```

① mov rsp,rbp; ② pop rbp

_sub:

```
push rbp;  
mov rbp rsp;  
...  
pop rbp;  
ret
```

保存上一个栈帧的顶部内存地址
rbp 指向当前栈帧的顶部位置

rbp 恢复到上一个栈帧顶部

指令系统：指令集体系结构 (ISA)

① 指令系统包含哪些内容

② 指令系统的设计原则

③ RISC 和 CISC 的区别

抽象：如何设计一个指令系统？



具体的指令系统实例：

MIPS

X86-64

三部分内容

寄存器组

指令格式

寻址方式

程序的机器级表示

选择结构

循环结构

函数调用

指令系统总结

① 指令格式

② 寻址方式

③ 指令系统设计基本问题

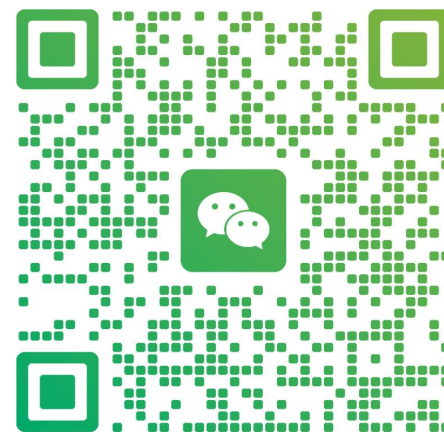
④ RISC 和 CISC 的区别

**加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】**



老汤

浙江 杭州



扫一扫上面的二维码图案，加我为朋友。

指令系统总结：指令格式（指令长度问题）

问题：一条指令用多少二进制位表示？ 指令应尽量短

第一种：所有的指令的长度都相等（定长指令）

MIPS 指令系统，指令字长为 32 位

第二种：不同类型的指令的长度不同（变长指令）

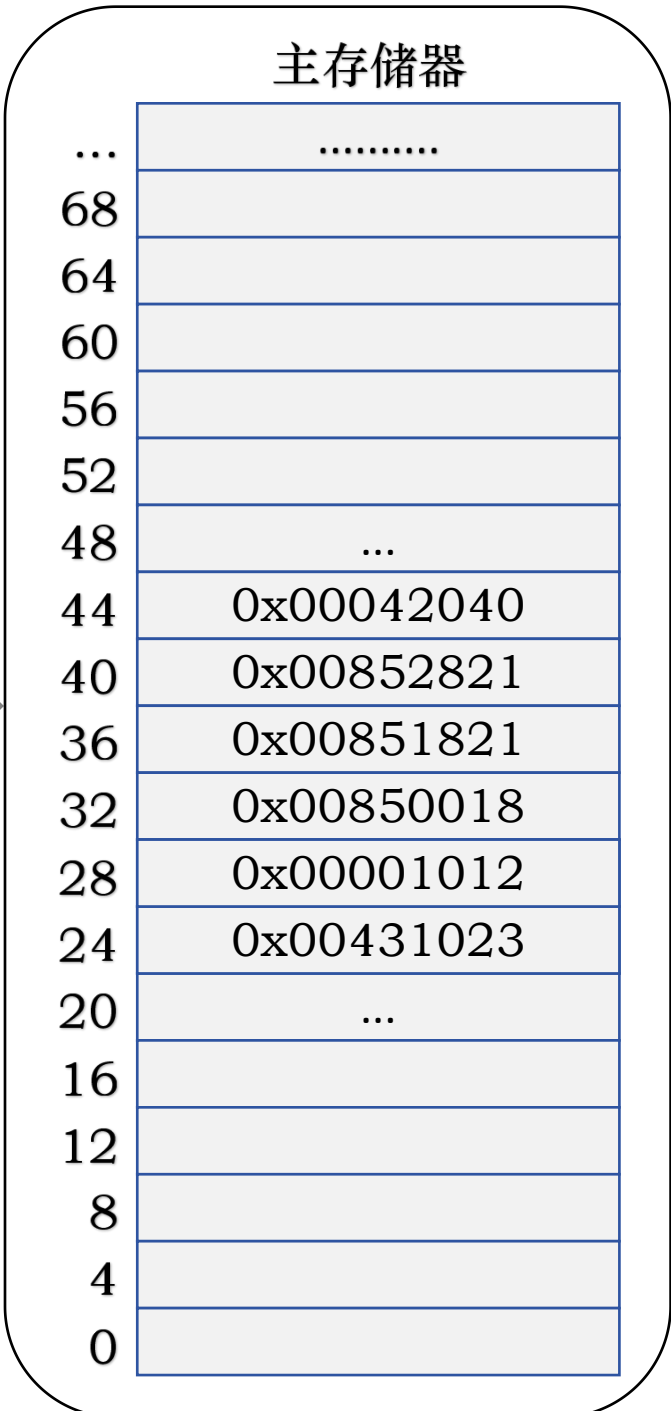
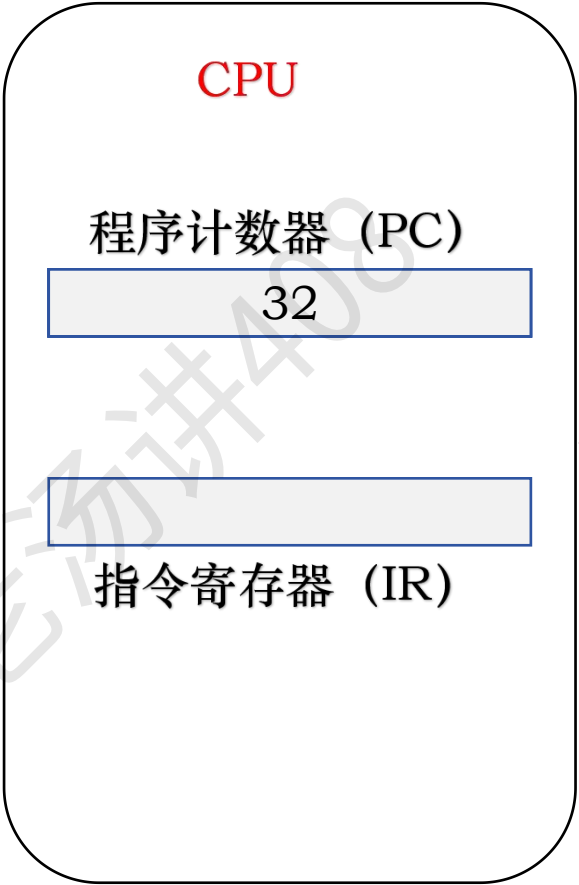
X86-64 指令系统，不同指令长度不同

MIPS 指令系统：指令格式 - 指令字长

```
#include<stdio.h>

int cal(int x, int n) {
    int a = x << 1;
    int b = a + n;
    int c = a + b;
    int d = a * b - c;
    return d;
}

int main() {
    int res = cal(9, 3);
    printf("res 的值是: %d\n", res);
    return 0;
}
```



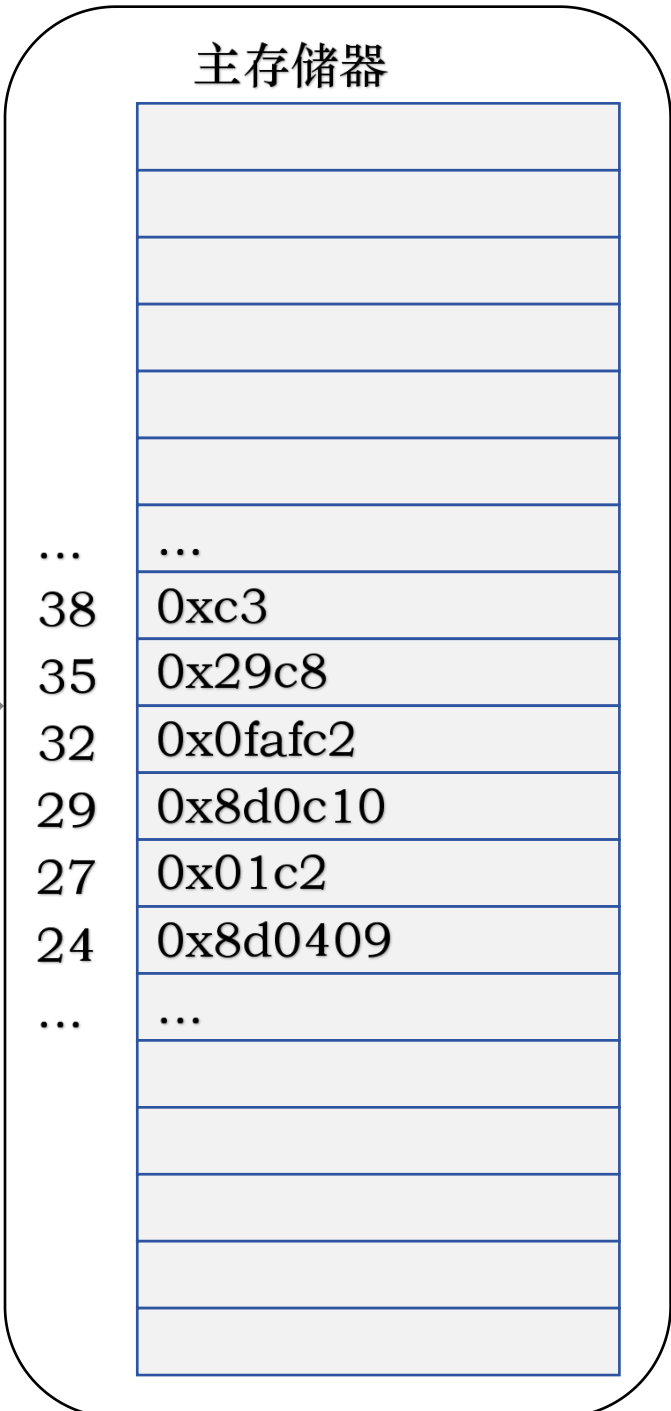
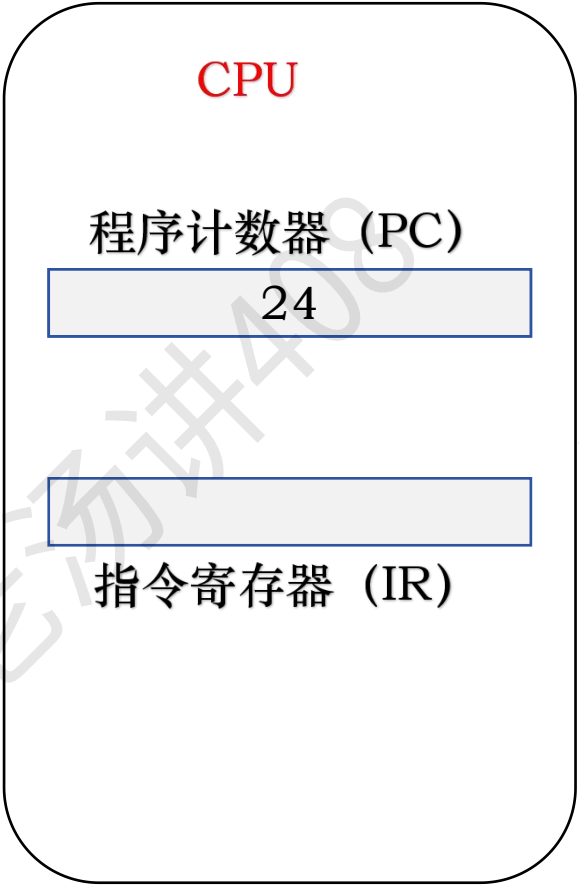
指令设计原则：简单源于规整

MIPS 指令系统：指令格式 - 指令字长

```
#include<stdio.h>

int cal(int x, int n) {
    int a = x << 1;
    int b = a + n;
    int c = a + b;
    int d = a * b - c;
    return d;
}

int main() {
    int res = cal(9, 3);
    printf("res 的值是: %d\n", res);
    return 0;
}
```



在 x86-64 中，每条指令的长度从 1 字节到 15 字节不等

MIPS 指令系统：指令格式 - 指令字长

```
#include<stdio.h>

int cal(int x, int n) {
    int a = x << 1;
    int b = a + n;
    int c = a + b;
    int d = a * b - c;
    return d;
}

int main() {
    int res = cal(9, 3);
    printf("res 的值是: %d\n", res);
    return 0;
}
```

MIPS 指令（指令长度不变）

0x00042040
0x00852821
0x00851821
0x00850018
0x00001012
0x00431023

24 字节

x86 指令（指令长度可变）

0xc3
0x29c8
0x0fafc2
0x8d0c10
0x01c2
0x8d0409

14 字节

MIPS 指令系统：R-型指令格式

```
#include<stdio.h>
```

```
int cal(int x, int n) {  
    int a = x << 1;  
    int b = a + n;  
    int c = a + b;  
    int d = a * b - c;  
    return d;  
}  
  
int main() {  
    int res = cal(9, 3);  
    printf("res 的值是: %d\n", res);  
    return 0;  
}
```

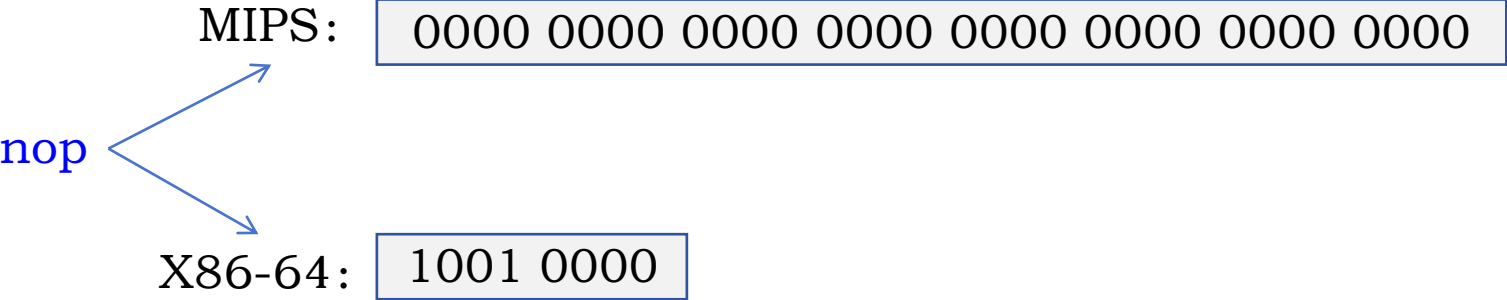


```
sll    $4,$4,1  
addu   $5,$4,$5  
addu   $3,$4,$5  
mult   $4,$5  
mflo   $2  
subu   $2,$2,$3
```

R 型指令格式:

op	rs	rt	rd	shamt	funct
----	----	----	----	-------	-------

000000	00000	00000	00010	00000	010010
--------	-------	-------	-------	-------	--------



指令系统总结：指令格式

操作码	地址码		地址码
-----	-----	-------	-----

操作码来表示指令的格式以及操作类型

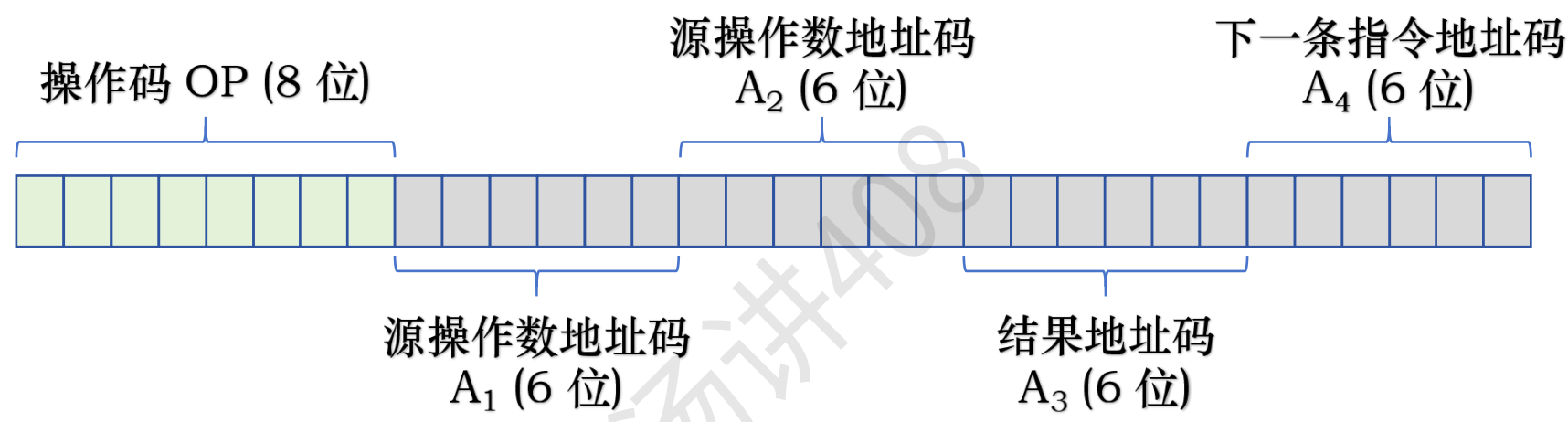
地址码用于指示操作数的位置，可以是立即数、寄存器编号、主存地址等

地址码的个数问题

操作码的编码问题

指令系统总结：指令格式（地址码个数问题）

① 四地址指令 指令字长：32 位



该指令完成的操作： $(A_1) \text{ OP } (A_2) \rightarrow A_3$

地址码可直接寻址的地址范围与地址码的位数有关

可寻址的地址范围为： $2^6 = 64$

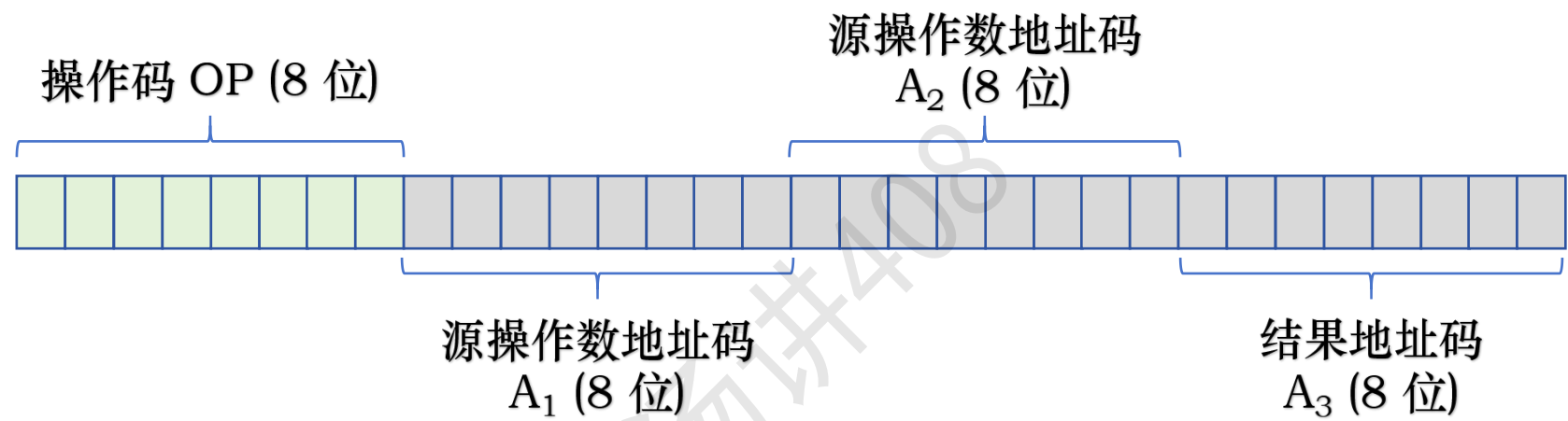
需要 4 次访存

- 取指令一次
- 取两个操作数两次
- 存放结果一次

指令系统总结：指令格式（地址码个数问题）

② 三地址指令 指令字长：32 位

下一条指令的地址：程序计数器 PC



该指令完成的操作： $(A_1) \text{ OP } (A_2) \rightarrow A_3$

可寻址的地址范围为： $2^8 = 256$

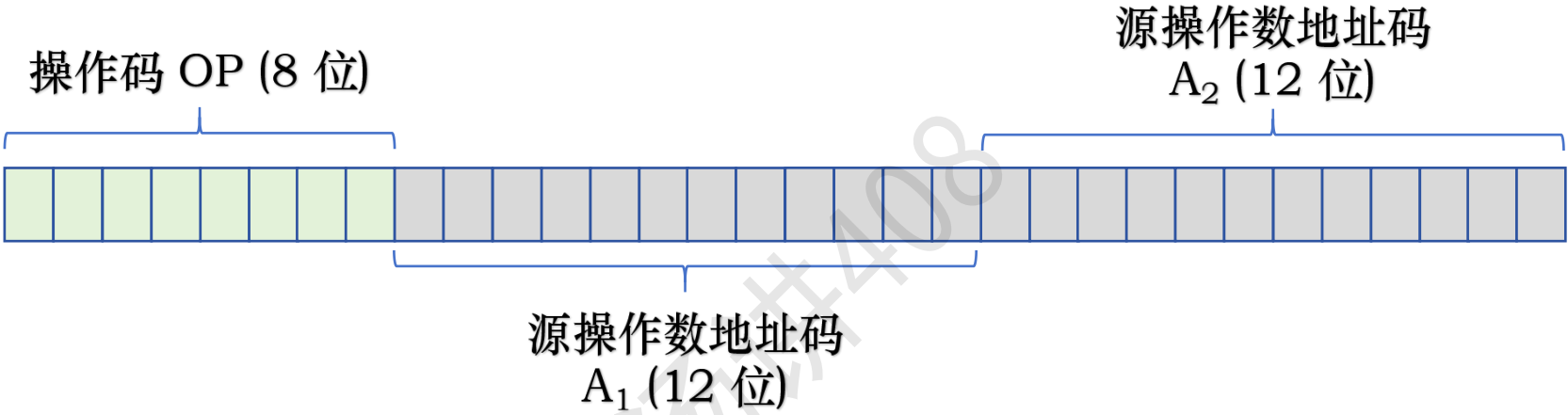
需要 4 次访存

- 取指令一次
- 取两个操作数两次
- 存放结果一次

指令系统总结：指令格式（地址码个数问题）

③ 二地址指令 指令字长：32 位

下一条指令的地址：程序计数器 PC



该指令完成的操作： $(A_1) \text{ OP } (A_2) \rightarrow A_1$ $(A_1) \text{ OP } (A_2) \rightarrow A_2$ $(A_1) \text{ OP } (A_2) \rightarrow \text{ACC}$

可寻址的地址范围为： $2^{12} = 4096 = 4\text{K}$

需要 4 次访存

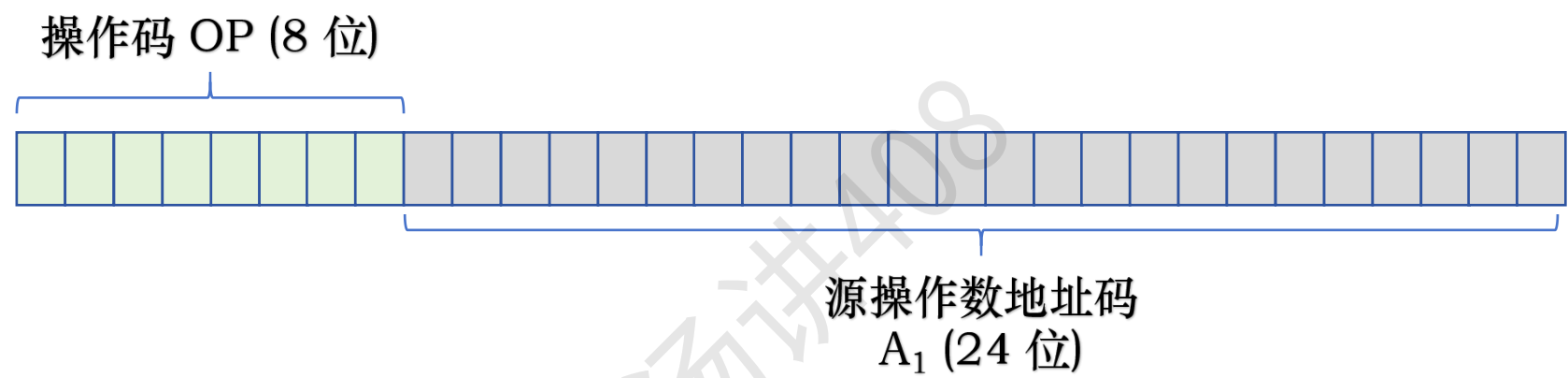
- 取指令一次
- 取两个操作数两次
- 存放结果一次 ❌

指令系统总结：指令格式（地址码个数问题）

④ 一地址指令

指令字长：32 位

下一条指令的地址：程序计数器 PC



该指令完成的操作： $(ACC) OP (A_1) \rightarrow ACC$ 隐含寻址

可寻址的地址范围为： $2^{24} = 16M$

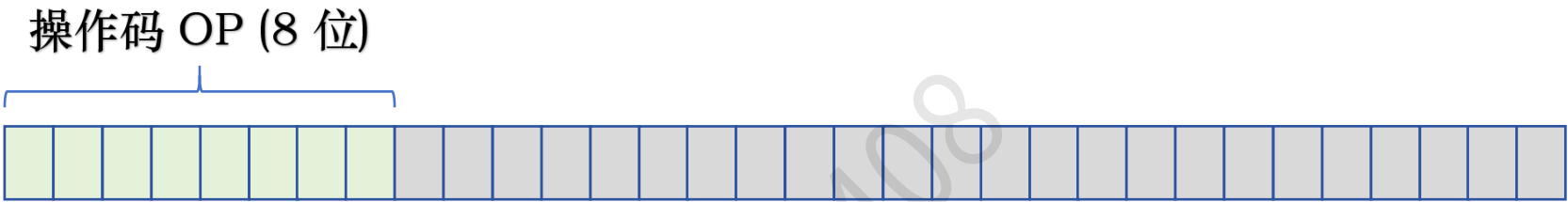
需要 2 次访存

- 取指令一次
- 取操作数一次

指令系统总结：指令格式（地址码个数问题）

⑤ 零地址指令 指令字长：32 位

下一条指令的地址：程序计数器 PC



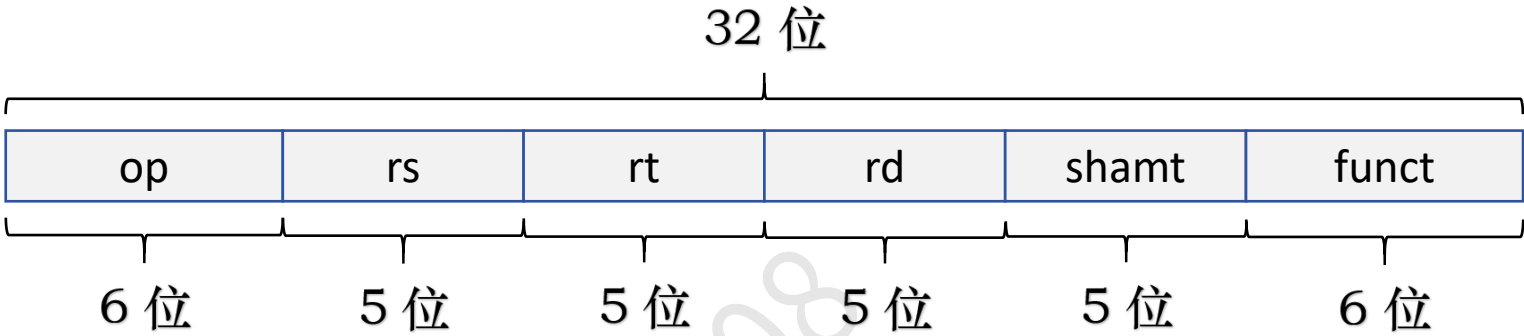
空操作 (NOP) 指令

停机指令 (HLT)

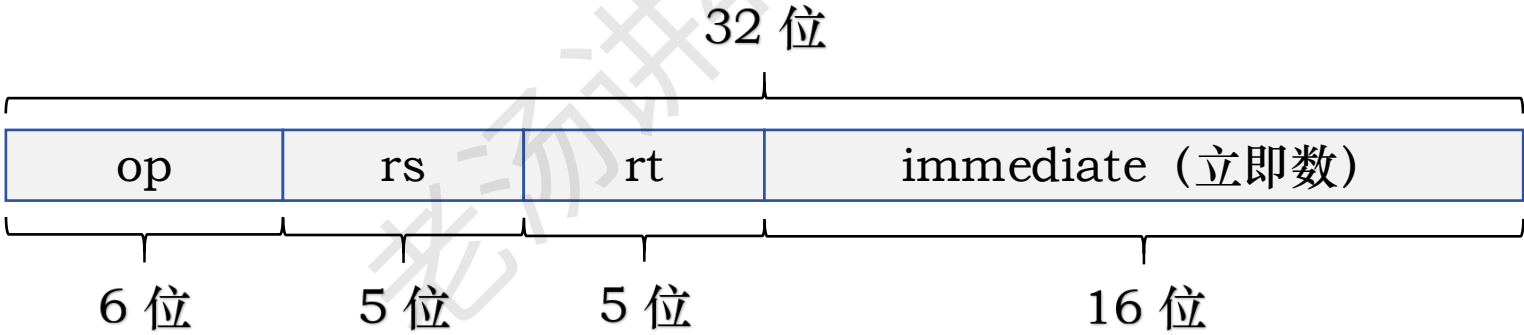
指令系统总结：指令格式（地址码个数问题） - MIPS

下一条指令的地址：程序计数器 PC

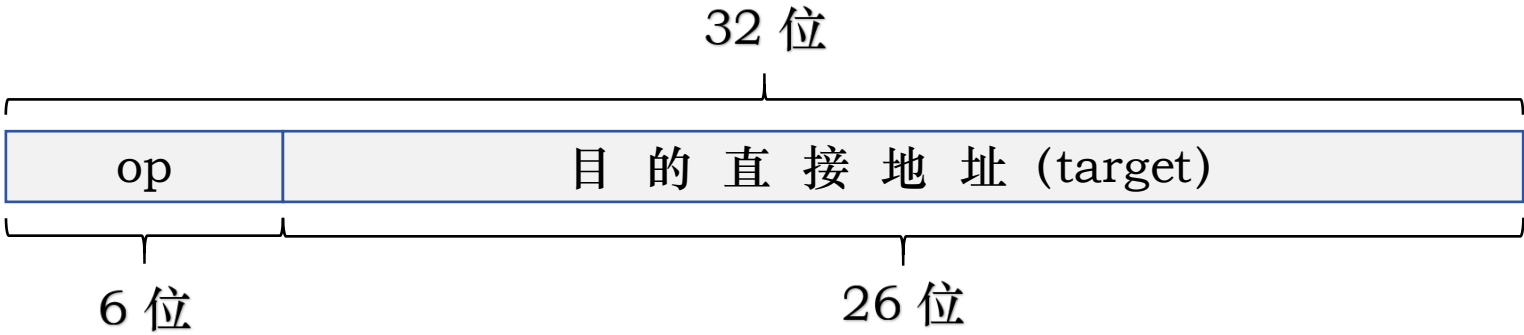
R-型指令格式



I-型指令格式



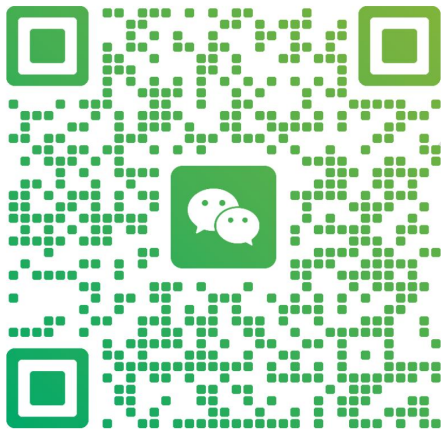
J-型指令格式



指令系统总结：指令格式（地址码个数问题）-X86-64

下一条指令的地址：程序计数器 PC（rip 寄存器）

加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】



扫一扫上面的二维码图案，加我为朋友。

```
fact_for:
    mov     eax, 2
    mov     edx, 1
    jmp     .L2
.L3:
    imul    rdx, rax
    add     rax, 1
.L2:
    cmp     rax, rdi
    jle     .L3
    mov     rax, rdx
    ret
fact_while:
    mov     eax, 1
    jmp     .L5
.L6:
    imul    rax, rdi
    sub     rdi, 1
.L5:
    cmp     rdi, 1
    jg      .L6
    ret
```

指令系统总结：指令格式（操作码编码）

操作码	地址码		地址码
-----	-----	-------	-----

IBM 360/370 采用 8 位定长操作码

优点：便于硬件设计、指令译码时间短 定长操作码

缺点：存在信息冗余

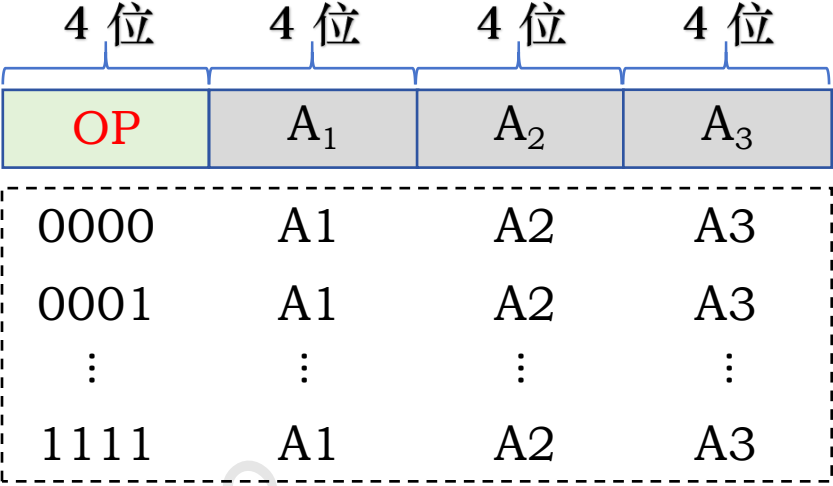
时间和空间之间的开销权衡

可以有效的压缩操作码的平均长度 变长操作码

采用扩展操作码技术实现 操作码的长度随着地址码数量的减少而增加

指令系统总结：指令格式（操作码编码）

指令字长：16 位



4 位操作码

16 条三地址指令

加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】



扫一扫上面的二维码图案，加我为朋友。

指令系统总结：指令格式（操作码编码）

指令格式 (操作码编码)					
	4 位	4 位	4 位	4 位	
	OP	A ₁	A ₂	A ₃	
4 位操作码	0000	A ₁	A ₂	A ₃	15 条三地址指令
	0001	A ₁	A ₂	A ₃	
	⋮	⋮	⋮	⋮	
	1110	A ₁	A ₂	A ₃	
8 位操作码	1111	0000	A ₂	A ₃	15 条二地址指令
	1111	0001	A ₂	A ₃	
	⋮	⋮	⋮	⋮	
	1111	1110	A ₂	A ₃	
12 位操作码	1111	1111	0000	A ₃	15 条一地址指令
	1111	1111	0001	A ₃	
	⋮	⋮	⋮	⋮	
	1111	1111	1110	A ₃	
16 位操作码	1111	1111	1111	0000	16 条零地址指令
	1111	1111	1111	0001	
	⋮	⋮	⋮	⋮	
	1111	1111	1111	1111	
总共 61 条指令					

指令系统总结：指令格式（操作码编码）

指令字长：16 位

总共 74 条指令

4 位	4 位	4 位	4 位
OP	A ₁	A ₂	A ₃
0000	A1	A2	A3
0001	A1	A2	A3
⋮	⋮	⋮	⋮
1110	A1	A2	A3
1111	0000	A2	A3
1111	0001	A2	A3
⋮	⋮	⋮	⋮
1111	1011	A2	A3
1111	1101	0000	A3
1111	1101	0001	A3
⋮	⋮	⋮	⋮
1111	1101	1110	A3
1111	1101	1111	0000
1111	1101	1111	0001
⋮	⋮	⋮	⋮
1111	1101	1111	1111

4 位操作码

15 条三地址指令

8 位操作码

12 条二地址指令

12 位操作码

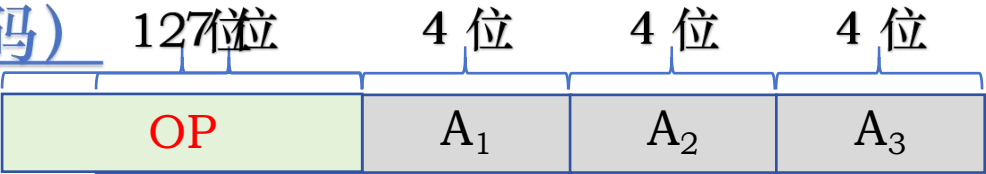
31 条一地址指令

16 位操作码

16 条零地址指令

1111	1100	0000	A3
1111	1100	0001	A3
⋮	⋮	⋮	⋮
1111	1100	1111	A3

指令系统总结：指令格式（操作码编码）



指令字长：24 位

定长操作码编码方式

$2^7 = 128 > 74$

扩展操作码编码可以有效的缩短指令字长

15 条三地址指令

12 条二地址指令

31 条一地址指令

总共 74 条指令

16 条零地址指令

指令系统总结：指令格式（操作码编码）

导致增加指令译码和分析的难度，使得 CPU 中的控制器的设计变得复杂

指令系统总结：指令格式（操作码编码）

例：假设某指令系统的指令字长是 16 位，每个地址码是 6 位，若二地址指令 15 条，单地址指令 34 条，则剩下的零地址指令最多有多少条？

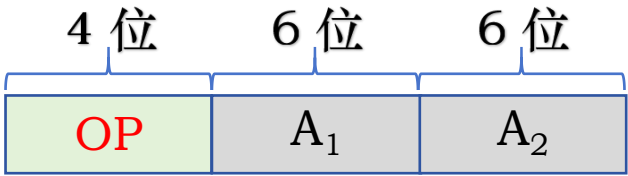
00010 ~ 11111 30

000000 ~ 111111 2⁶

4 位操作码

10 位操作码

16 位操作码



0000	A1	A2
0001	A1	A2
⋮	⋮	⋮
1110	A1	A2

15 条二地址指令

1111	000000	A2
1111	000001	A2
⋮	⋮	⋮
1111	011111	A2
1111	100000	A2
1111	100001	A2

34 条一地址指令

1111	100010	000000
1111	100010	000000
⋮	⋮	⋮
1111	111111	111111

30 × 2⁶ 零地址指令

指令系统总结：寻址方式

① 立即数寻址

② 直接寻址

③ 间接寻址

④ 相对寻址

偏移寻址 { ⑤ 基址寻址

⑥ 变址寻址

⑦ 隐含寻址

堆栈寻址

ACC 累加器

指令系统总结：寻址方式 (① 立即数寻址)

在指令中直接给出操作数本身 `sub rdx, 100`

这样指令执行速度虽然很快，

但是这个立即数很可能很大，导致指令长度变长

```
int res = 400000;
```

3c050006

`lui    a1,0x6`

24a51a80

`addiu   a1,a1,6784`

MIPS

大端序

be 80 1a 06 00

`mov    esi,0x61a80`

X86-64

小端序

X86-64 指令系统：寻址方式 (① 直接寻址)

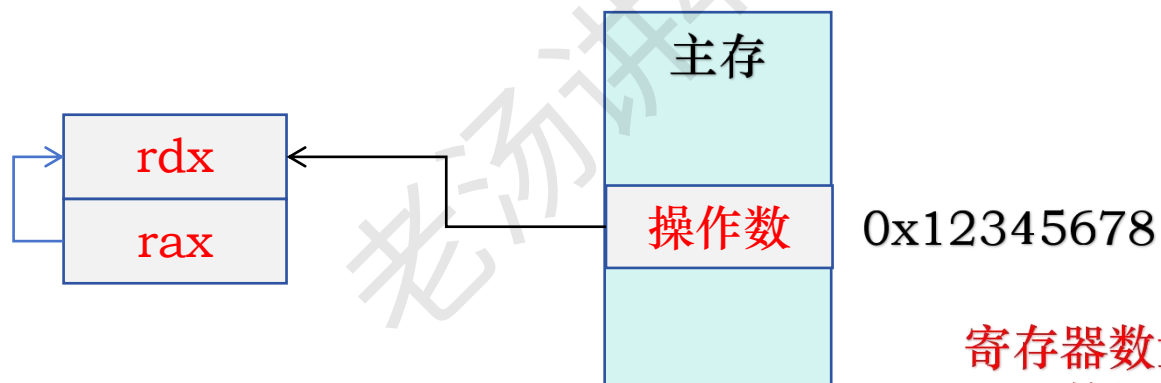
① 寄存器数量远小于主存单元数，所以寄存器编号比主存地址要短很多，因而寄存器寻址方式的指令比较短

寄存器直接寻址：操作数在寄存器中

`mov rdx, rax`

主存直接寻址：操作数在主存中

`mov rdx, [0x12345678]`



寄存器数量有限，容量有限，很可能存不下数据，这时数据就要存储到主存中

`sub rdx, rax`

`sub rdx, [0x12345678]`

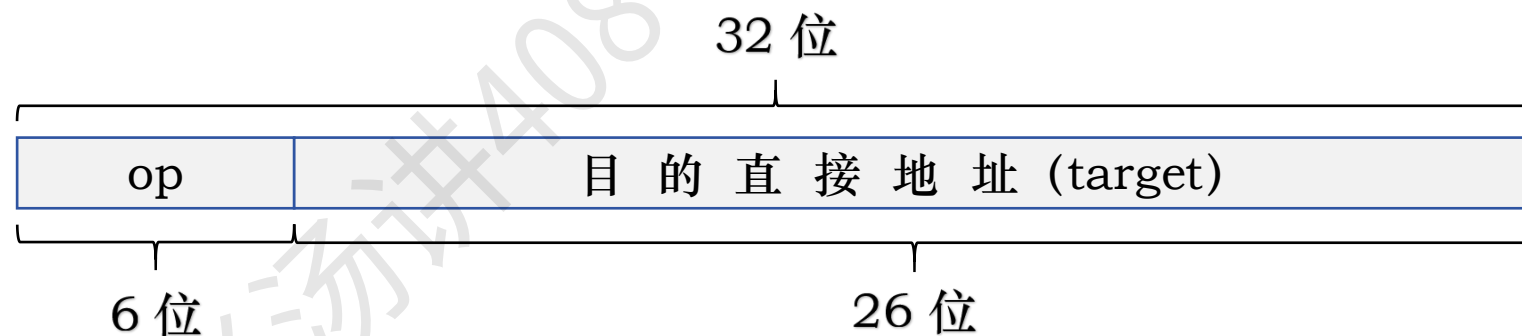
② 操作数已经在 CPU 中，不用访问主存，因而指令执行速度快

X86-64 指令系统：寻址方式 (① 直接寻址)

寄存器直接寻址：操作数在寄存器中

主存直接寻址：操作数在主存中

J-型指令格式

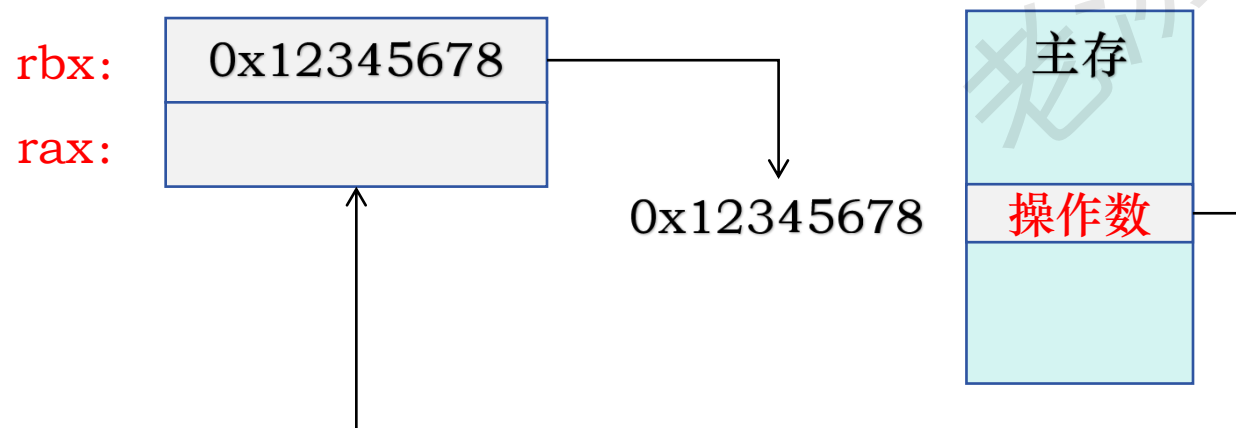


x86-64: `mov rdx, [0x12345678]`

X86-64 指令系统：寻址方式 (③ 间接寻址)

寄存器间接寻址：指令中给出的地址码是一个寄存器编号，该寄存器中存放的是操作数的有效地址

```
mov rax, [rbx]
```



加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】



老汤
浙江 杭州



扫一扫上面的二维码图案，加我为朋友。

X86-64 支持寄存器间接寻址

```
          rdi      rsi
          ↑        ↑
void cond(long a, long *p) {
    if (p && a > *p)
        *p = a;
}
```

cond:

```
test    rsi, rsi
```

```
je      .L1
```

```
cmp     QWORD PTR [rsi], rdi
```

```
jge     .L1
```

```
mov     QWORD PTR [rsi], rdi
```

.L1:

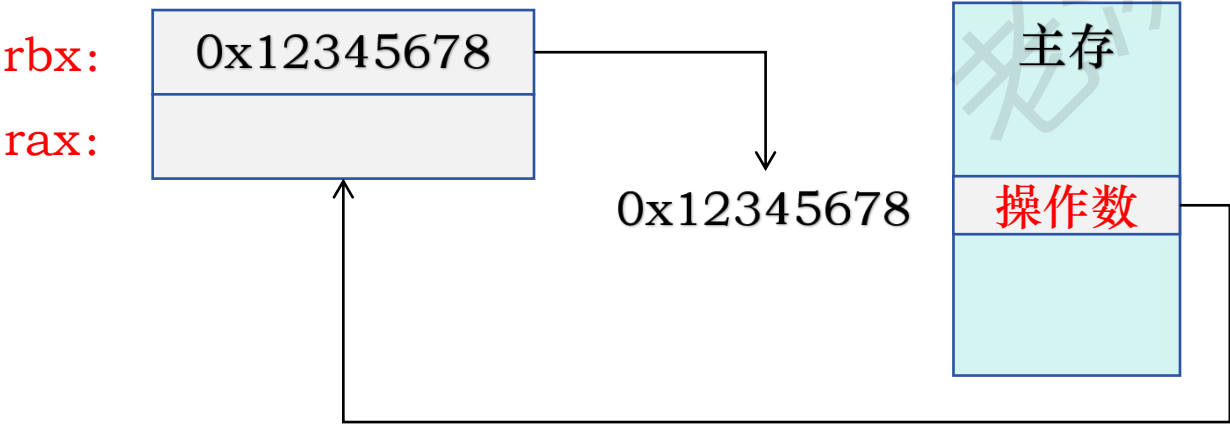
```
ret
```

X86-64 指令系统：寻址方式 (③ 间接寻址)

MIPS 不支持、Intel x86-64 支持

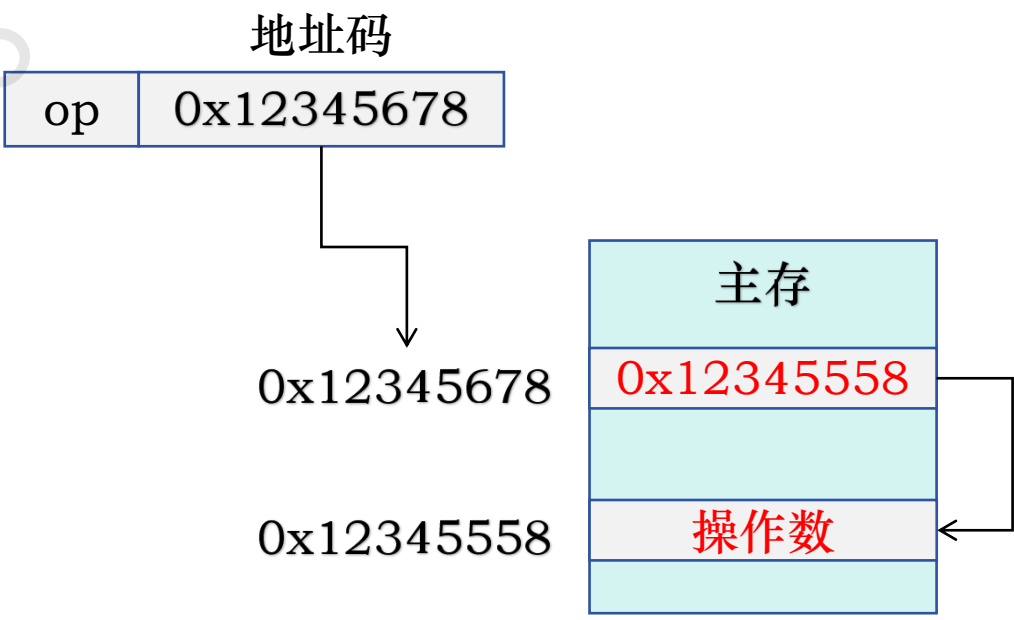
寄存器间接寻址：指令中给出的地址码是一个寄存器编号，该寄存器中存放的是操作数的有效地址

```
mov rax, [rbx]
```

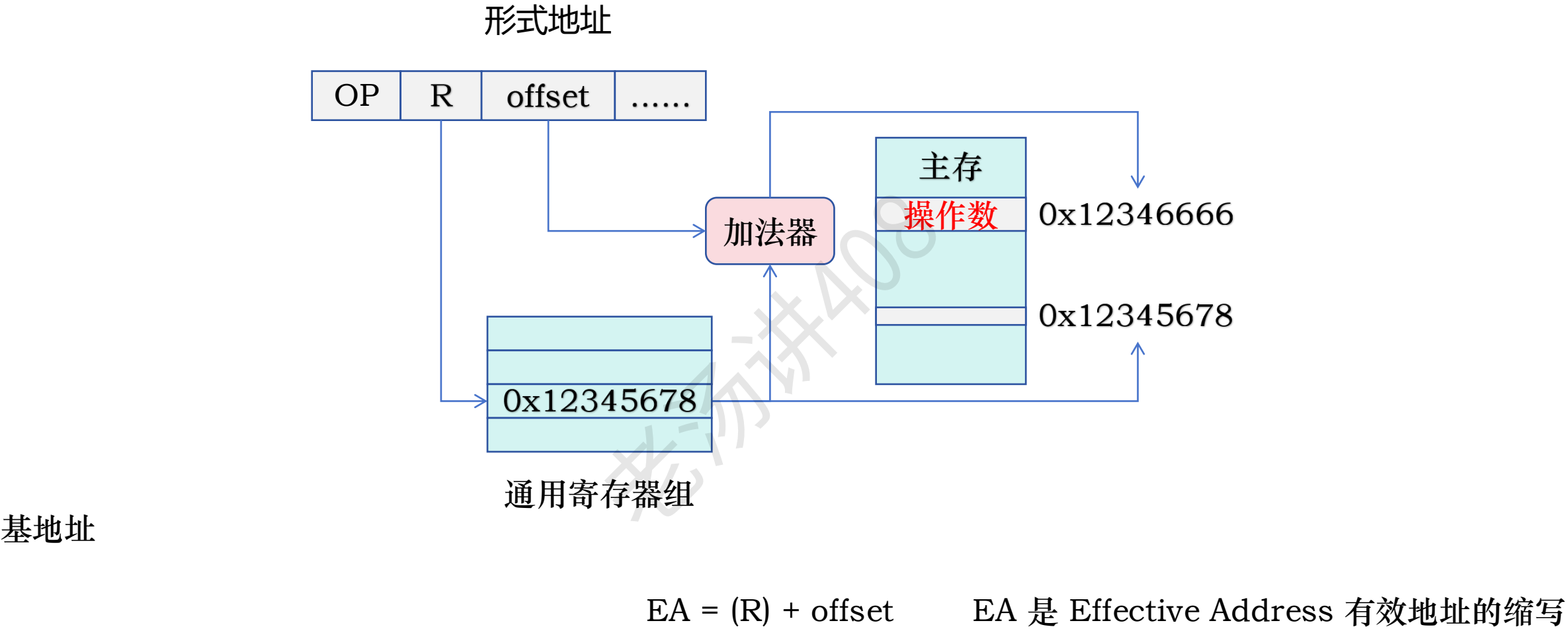


Intel x86-64 、MIPS 都不支持

主存间接寻址：指令中给出的地址码是存放操作数有效地址的主存单元地址



X86-64 指令系统：寻址方式 (④ 基址寻址)

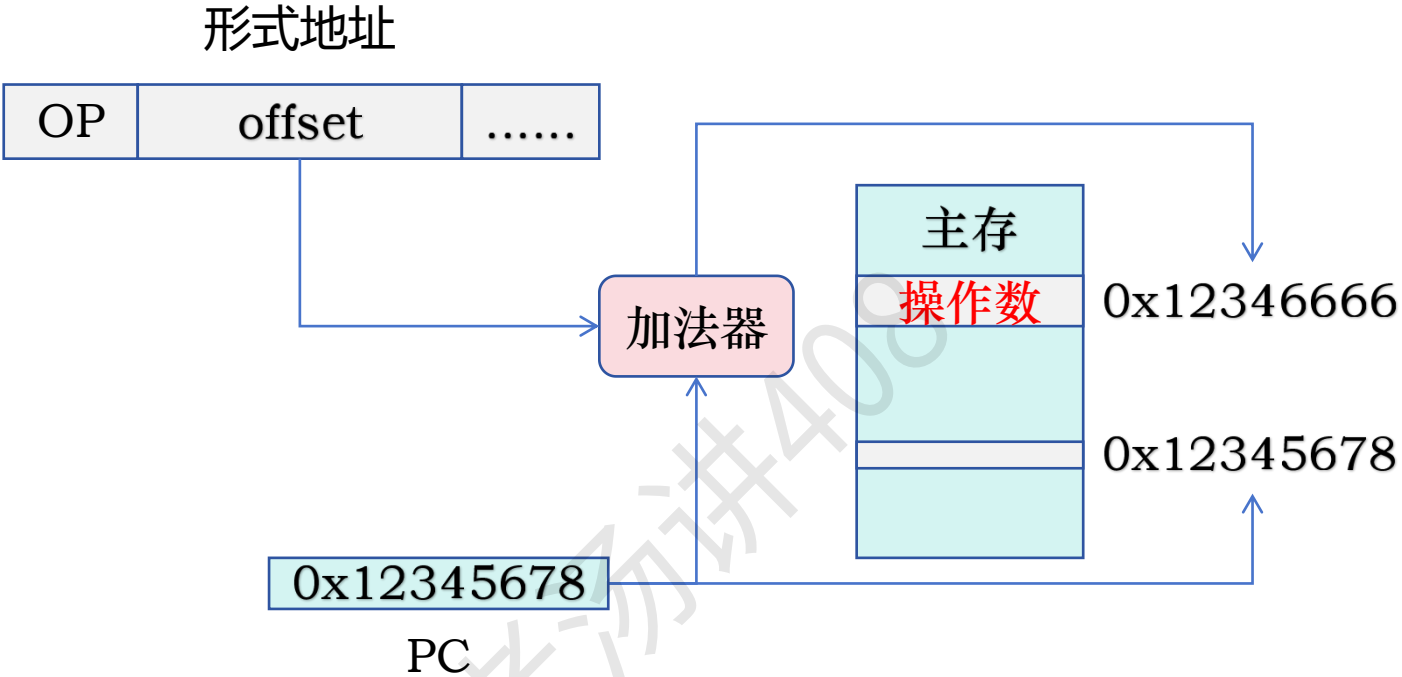


基地址

X86-64: `mov rax, QWORD PTR [rbp-24]`

MIPS: `sw $31, 28($sp)`

X86-64 指令系统：寻址方式 (⑤ 相对寻址)



$EA = (PC) + offset$

X86-64: `jmp QWORD PTR [rip+0x200c34]`

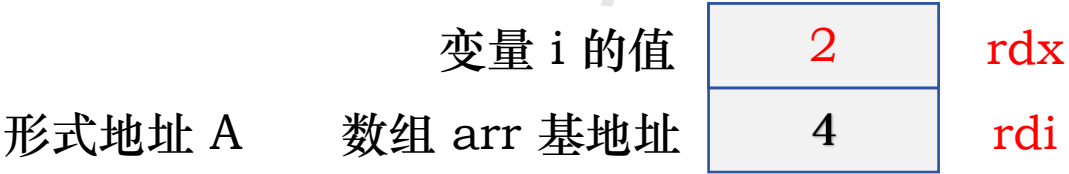
MIPS: `beq $2,$0,8`

X86-64 指令系统：寻址方式 (⑥ 变址寻址)

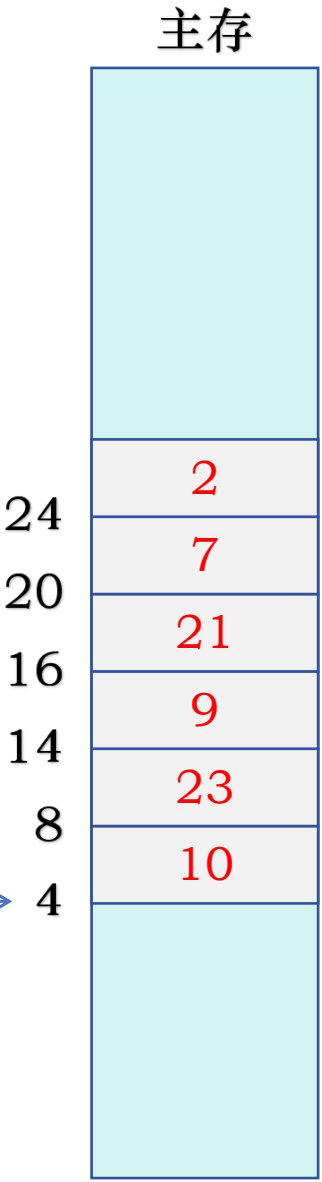
```
int sum_arr(int arr[], int n) {  
    int res = 0;  
    for (int i = 0; i < n; i++) {  
        res += arr[i];  
    }  
    return res;  
}
```

比例变址寻址方式

add eax, DWORD PTR [rdi+rdx*4]



EA = (I) + A



X86-64 指令系统：寻址方式 (⑥ 变址寻址)

基址寻址: $EA = (R) + offset$

相对寻址: $EA = (PC) + offset$

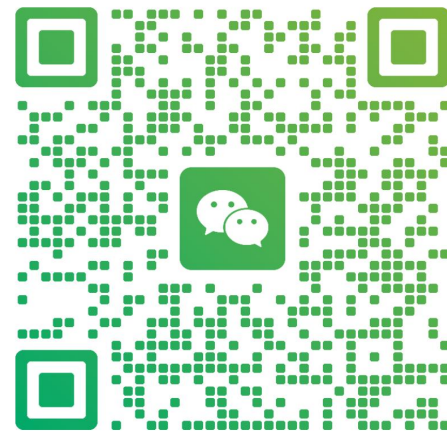
变址寻址: $EA = (I) + A$

加老汤微信，帮你 408 冲 120+



老汤

浙江 杭州



偏移寻址

扫一扫上面的二维码图案，加我为朋友。

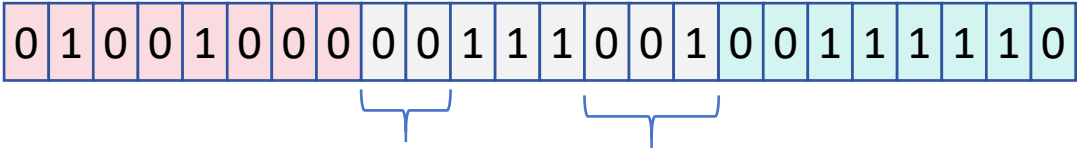
X86-64 指令系统：寻址方式

指令：

操作码	寻址方式	形式地址 A	寻址方式	形式地址 A
-----	------	--------	------	--------

加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】

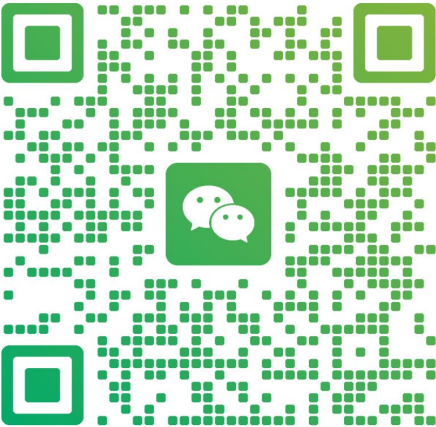
X86-64: 48 39 3e
`cmp      QWORD PTR [rsi], rdi`



寄存器间接寻址



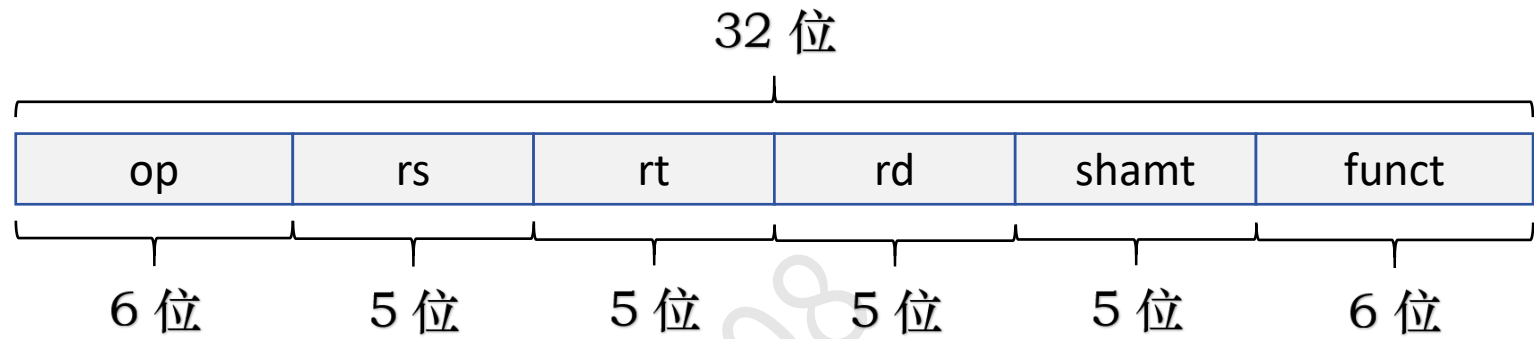
老汤
浙江 杭州



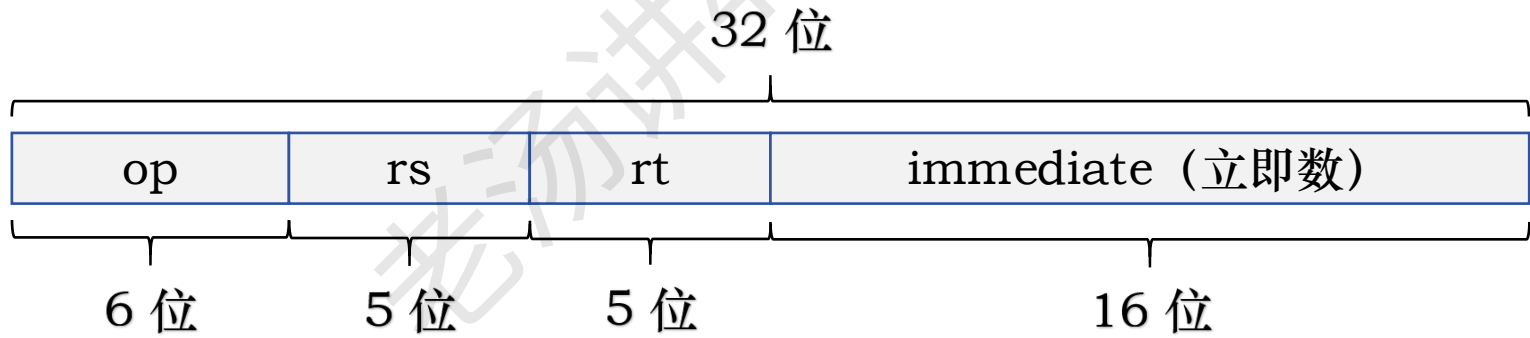
扫一扫上面的二维码图案，加我为朋友。

指令系统总结：指令格式 - MIPS

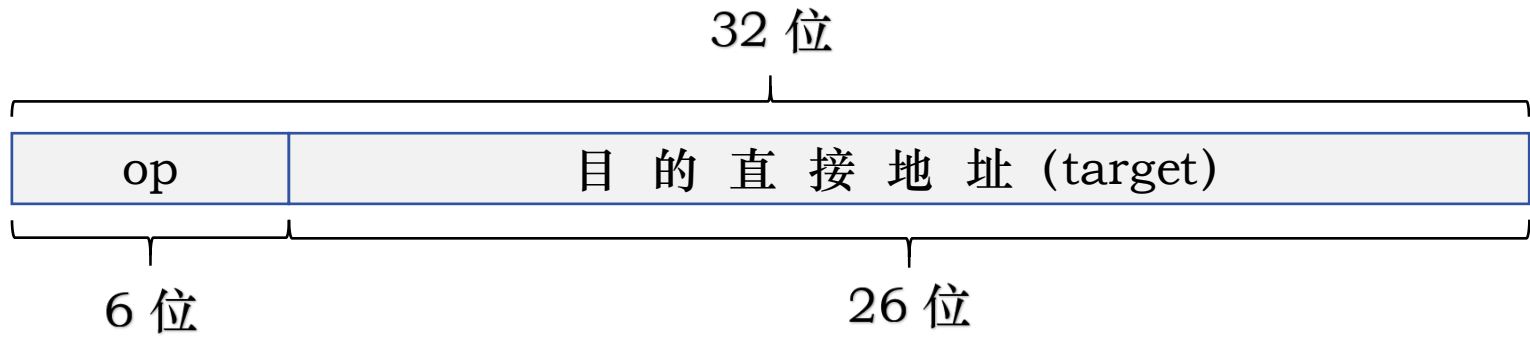
R-型指令格式



I-型指令格式



J-型指令格式



指令系统：寻址方式（变址寻址 + 间址寻址）

变址寄存器 I: 4

形式地址: $A = 4$

先变址再间址: $EA = ((I) + A)$

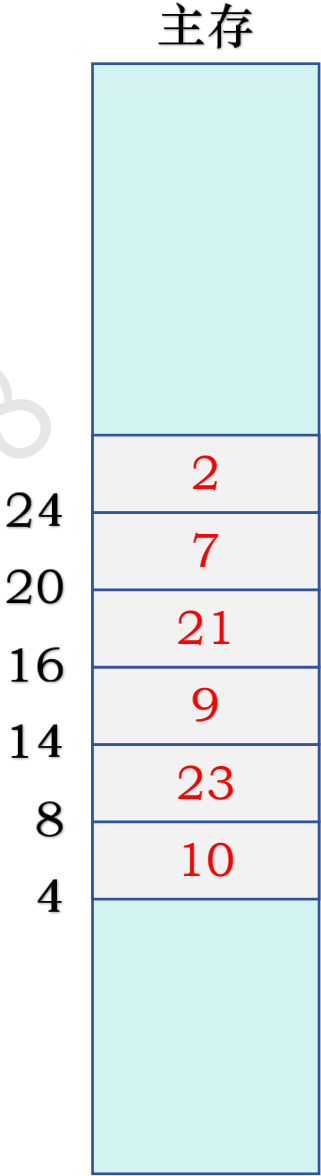
先变址, 操作数有效地址为 $4 + 4 = 8$

再间址, 操作数有效地址为内存地址为 8 的位置数据, 即 23

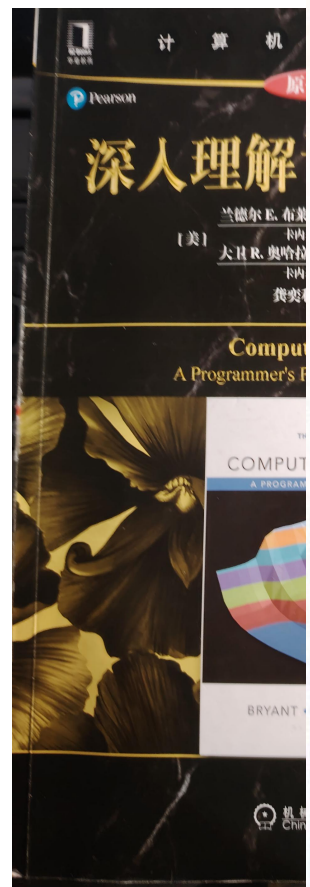
先间再变址: $EA = (A) + (I)$

先间址, 操作数有效地址为 10

再变址, 操作数有效地址为 $10 + 4 = 14$



指令系统总



CISC	早期的 RISC
指令数量很多。Intel 描述全套指令的文档[51]有 1200 多页。	指令数量少得多。通常少于 100 个。
有些指令的延迟很长。包括将一个整块从内存的一个部分复制到另一部分的指令，以及其他一些将多个寄存器的值复制到内存或从内存复制到多个寄存器的指令。	没有较长延迟的指令。有些早期的 RISC 机器甚至没有整数乘法指令，要求编译器通过一系列加法来实现乘法。
编码是可变长度的。x86-64 的指令长度可以是 1~15 个字节。	编码是固定长度的。通常所有的指令都编码为 4 个字节。
指定操作数的方式很多样。在 x86-64 中，内存操作数指示符可以有许多不同的组合，这些组合由偏移量、基址和变址寄存器以及伸缩因子组成。	简单寻址方式。通常只有基址和偏移量寻址。
可以对内存和寄存器操作数进行算术和逻辑运算。	只能对寄存器操作数进行算术和逻辑运算。允许使用内存引用的只有 load 和 store 指令，load 是从内存读到寄存器，store 是从寄存器写到内存。这种方法被称为 load/store 体系结构。
对机器级程序来说实现细节是不可见的。ISA 提供了程序和如何执行程序之间的清晰的抽象。	对机器级程序来说实现细节是可见的。有些 RISC 机器禁止某些特殊的指令序列，而有些跳转要到下一条指令执行完了以后才会生效。编译器必须在这些约束条件下进行性能优化。
有条件码。作为指令执行的副产品，设置了一些特殊的标志位，可以用于条件分支检测。	没有条件码。相反，对条件检测来说，要用明确的测试指令，这些指令会将测试结果放在一个普通的寄存器中。
栈密集的过程链接。栈被用来存取过程参数和返回地址。	寄存器密集的过程链接。寄存器被用来存取过程参数和返回地址。因此有些过程能完全避免内存引用。通常处理器有更多的(最多的有 32 个)寄存器。

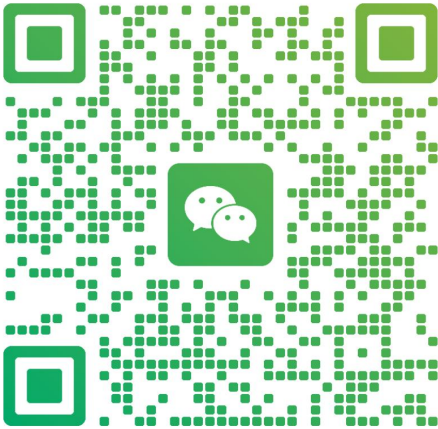
指令系统总结: RISC vs CISC

指令	同义名	跳转条件	描述
jmp <i>Label</i>		1	直接跳转
jmp <i>*Operand</i>		1	间接跳转
jz <i>Label</i>	jz	ZF	相等/零
jne <i>Label</i>	jnz	~ZF	不相等/非零
js <i>Label</i>		SF	负数
jns <i>Label</i>		~SF	非负数
jg <i>Label</i>	jnle	~(SF ^ OF) & ~ZF	大于 (有符号>)
jge <i>Label</i>	jnl	~(SF ^ OF)	大于或等于 (有符号>=)
jl <i>Label</i>	jnge	SF ^ OF	小于 (有符号<)
jle <i>Label</i>	jng	(SF ^ OF) ZF	小于或等于 (有符号<=)
ja <i>Label</i>	jnbe	~CF & ~ZF	超过 (无符号>)
jae <i>Label</i>	jnb	~CF	超过或相等 (无符号>=)
jb <i>Label</i>	jnae	CF	低于 (无符号<)
jbe <i>Label</i>	jna	CF ZF	低于或相等 (无符号<=)

加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】

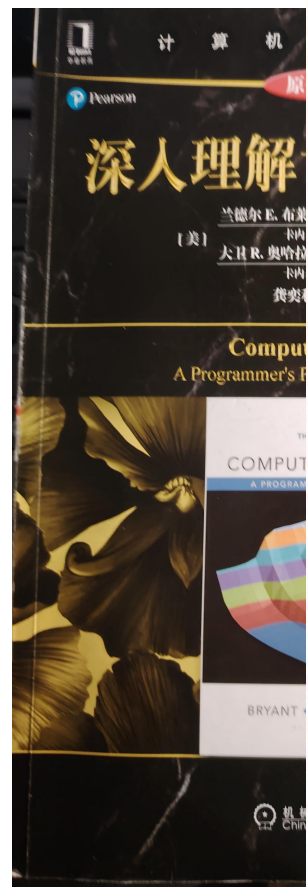


老汤
浙江 杭州



扫一扫上面的二维码图案，加我为朋友。

指令系统总



CISC	早期的 RISC
指令数量很多。Intel 描述全套指令的文档[51]有 1200 多页。	指令数量少得多。通常少于 100 个。
有些指令的延迟很长。包括将一个整块从内存的一个部分复制到另一部分的指令，以及其他一些将多个寄存器的值复制到内存或从内存复制到多个寄存器的指令。	没有较长延迟的指令。有些早期的 RISC 机器甚至没有整数乘法指令，要求编译器通过一系列加法来实现乘法。
编码是可变长度的。x86-64 的指令长度可以是 1~15 个字节。	编码是固定长度的。通常所有的指令都编码为 4 个字节。
指定操作数的方式很多样。在 x86-64 中，内存操作数指示符可以有许多不同的组合，这些组合由偏移量、基址和变址寄存器以及伸缩因子组成。	简单寻址方式。通常只有基址和偏移量寻址。
可以对内存和寄存器操作数进行算术和逻辑运算。	只能对寄存器操作数进行算术和逻辑运算。允许使用内存引用的只有 load 和 store 指令，load 是从内存读到寄存器，store 是从寄存器写到内存。这种方法被称为 load/store 体系结构。
对机器级程序来说实现细节是不可见的。ISA 提供了程序和如何执行程序之间的清晰的抽象。	对机器级程序来说实现细节是可见的。有些 RISC 机器禁止某些特殊的指令序列，而有些跳转要到下一条指令执行完了以后才会生效。编译器必须在这些约束条件下进行性能优化。
有条件码。作为指令执行的副产品，设置了一些特殊的标志位，可以用于条件分支检测。	没有条件码。相反，对条件检测来说，要用明确的测试指令，这些指令会将测试结果放在一个普通的寄存器中。
栈密集的过程链接。栈被用来存取过程参数和返回地址。	寄存器密集的过程链接。寄存器被用来存取过程参数和返回地址。因此有些过程能完全避免内存引用。通常处理器有更多的(最多的有 32 个)寄存器。

指令系统总结: RISC vs CISC

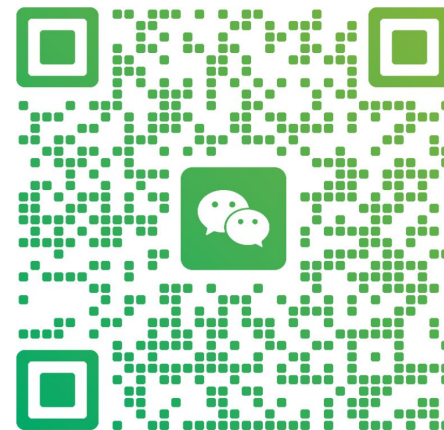
```
slt    $3,$4,2  
beq    $3,$0,$L3
```

加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】



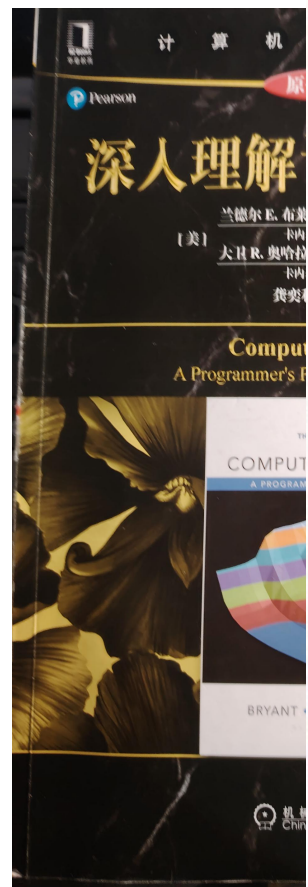
老汤

浙江 杭州



扫一扫上面的二维码图案，加我为朋友。

指令系统总



CISC	早期的 RISC
指令数量很多。Intel 描述全套指令的文档[51]有 1200 多页。	指令数量少得多。通常少于 100 个。
有些指令的延迟很长。包括将一个整块从内存的一个部分复制到另一部分的指令，以及其他一些将多个寄存器的值复制到内存或从内存复制到多个寄存器的指令。	没有较长延迟的指令。有些早期的 RISC 机器甚至没有整数乘法指令，要求编译器通过一系列加法来实现乘法。
编码是可变长度的。x86-64 的指令长度可以是 1~15 个字节。	编码是固定长度的。通常所有的指令都编码为 4 个字节。
指定操作数的方式很多样。在 x86-64 中，内存操作数指示符可以有许多不同的组合，这些组合由偏移量、基址和变址寄存器以及伸缩因子组成。	简单寻址方式。通常只有基址和偏移量寻址。
可以对内存和寄存器操作数进行算术和逻辑运算。	只能对寄存器操作数进行算术和逻辑运算。允许使用内存引用的只有 load 和 store 指令，load 是从内存读到寄存器，store 是从寄存器写到内存。这种方法被称为 load/store 体系结构。
对机器级程序来说实现细节是不可见的。ISA 提供了程序和如何执行程序之间的清晰的抽象。	对机器级程序来说实现细节是可见的。有些 RISC 机器禁止某些特殊的指令序列，而有些跳转要到下一条指令执行完了以后才会生效。编译器必须在这些约束条件下进行性能优化。
有条件码。作为指令执行的副产品，设置了一些特殊的标志位，可以用于条件分支检测。	没有条件码。相反，对条件检测来说，要用明确的测试指令，这些指令会将测试结果放在一个普通的寄存器中。
栈密集的过程链接。栈被用来存取过程参数和返回地址。	寄存器密集的过程链接。寄存器被用来存取过程参数和返回地址。因此有些过程能完全避免内存引用。通常处理器有更多的(最多的有 32 个)寄存器。

指令系统总结：指令格式设计原则

① 指令应尽量短

② 要有足够的操作码位数

③ 操作码的编码必须有唯一的解释

当译码器发现是不合法操作码时，出现“非法指令”异常

④ 指令长度应是字节的整数倍

⑤ 合理选择地址码的个数

⑥ 指令应尽量规整